SentinelView: Full End-to-End SOC Deployment & Adversary Emulation Report

Project Name: SentinelView
Objective: Deploy a complete Security Information and Event Management (SIEM) pipeline on AWS, execute MITRE ATT&CK adversarial emulations across Linux and Windows endpoints, and visualize the threats in real-time through a custom AI-integrated React dashboard.

---

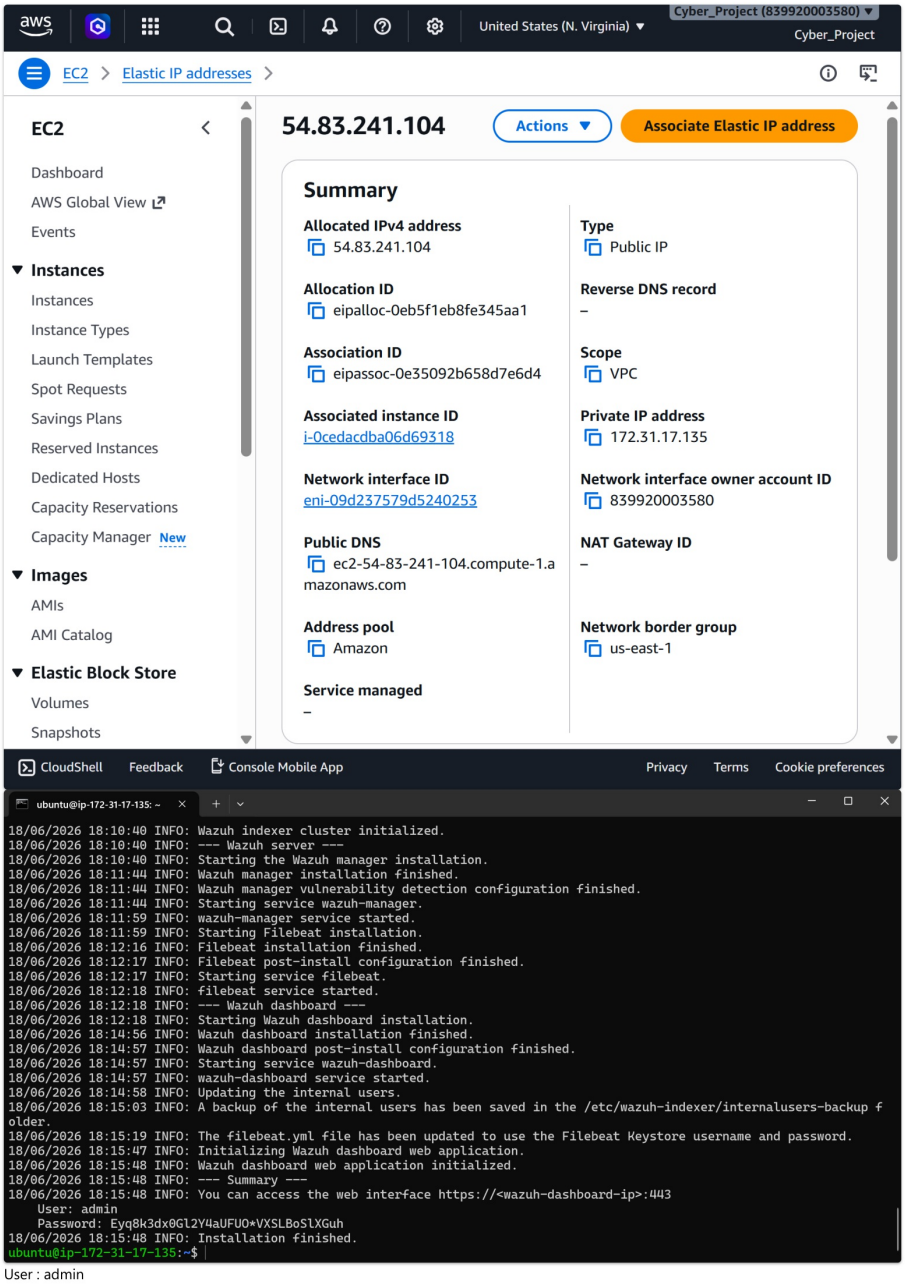
1. Cloud Engineering & Infrastructure (Hazem & Abdelrahman)

The foundation of the project relies on a robust and secure Cloud Infrastructure hosted on AWS. Hazem and Abdelrahman engineered the deployment of the Wazuh Central Manager and its routing components.

1.1 AWS EC2 Provisioning

1. **Launch Instance:** Navigated to the AWS EC2 Console to provision the central server.

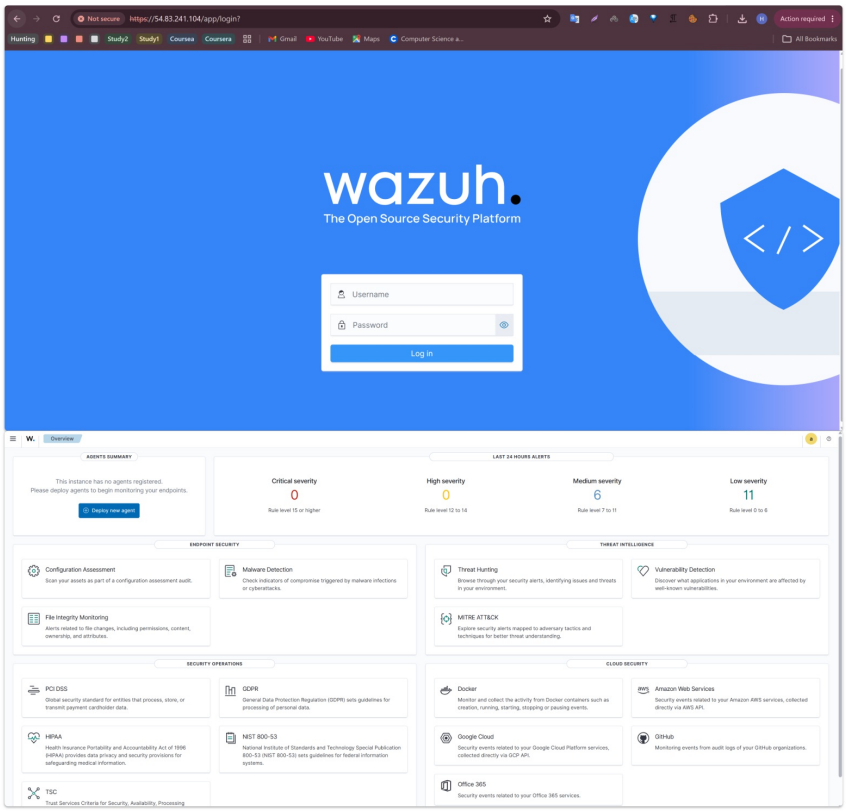
DEPI Project



User : admin
Password : Eyq8k3dx0Gl2Y4aUFUO*VXSLBoSlXGuh

2. **Instance Configuration:**
    - **Name:** `Wazuh-Manager`

- **OS:** Ubuntu Server 24.04 LTS
- **Instance Type:** `m7i-flex.large` (to handle high-volume Elasticsearch indexing)
- **Key Pair:** Generated `Wazuh-Key.pem` (RSA) for secure SSH access.
- **Storage:** Allocated a `40 GB GP3` EBS volume.



المرحلة 1: إنشاء EC2

1. ادخل EC2

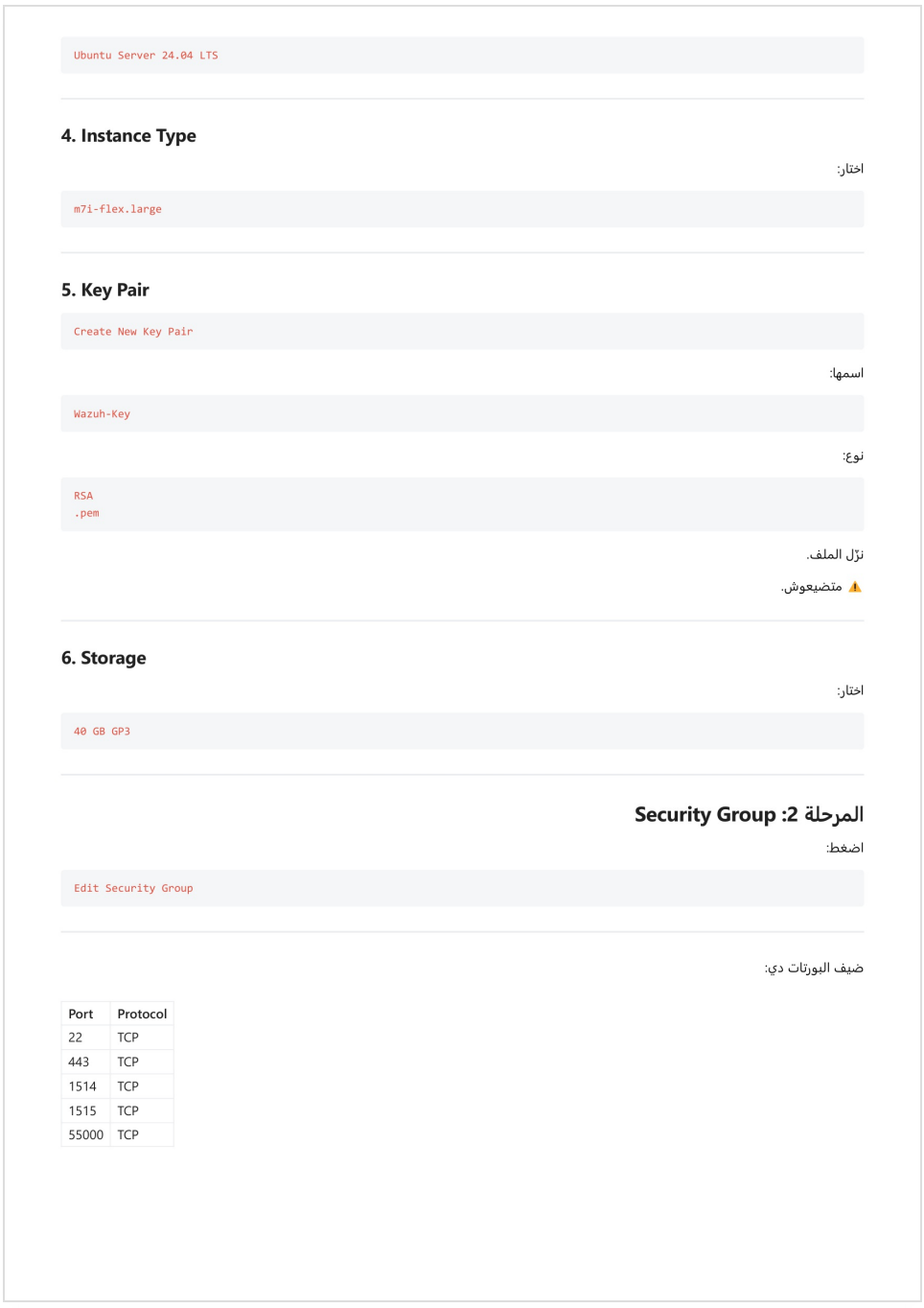
```
AWS Console
↓
EC2
↓
Launch Instance
```

2. اسم الـ Instance

```
Wazuh-Manager
```

3. اختار الـ OS

اختار:



```
Ubuntu Server 24.04 LTS
```

4. Instance Type

اختار:

```
m7i-flex.large
```

5. Key Pair

```
Create New Key Pair
```

اسمها:

```
Wazuh-Key
```

نوع:

```
RSA
.pem
```

نزّل الملف.

⚠️ متضيعوش.

6. Storage

اختار:

```
40 GB GP3
```

المرحلة 2: Security Group

اضغط:

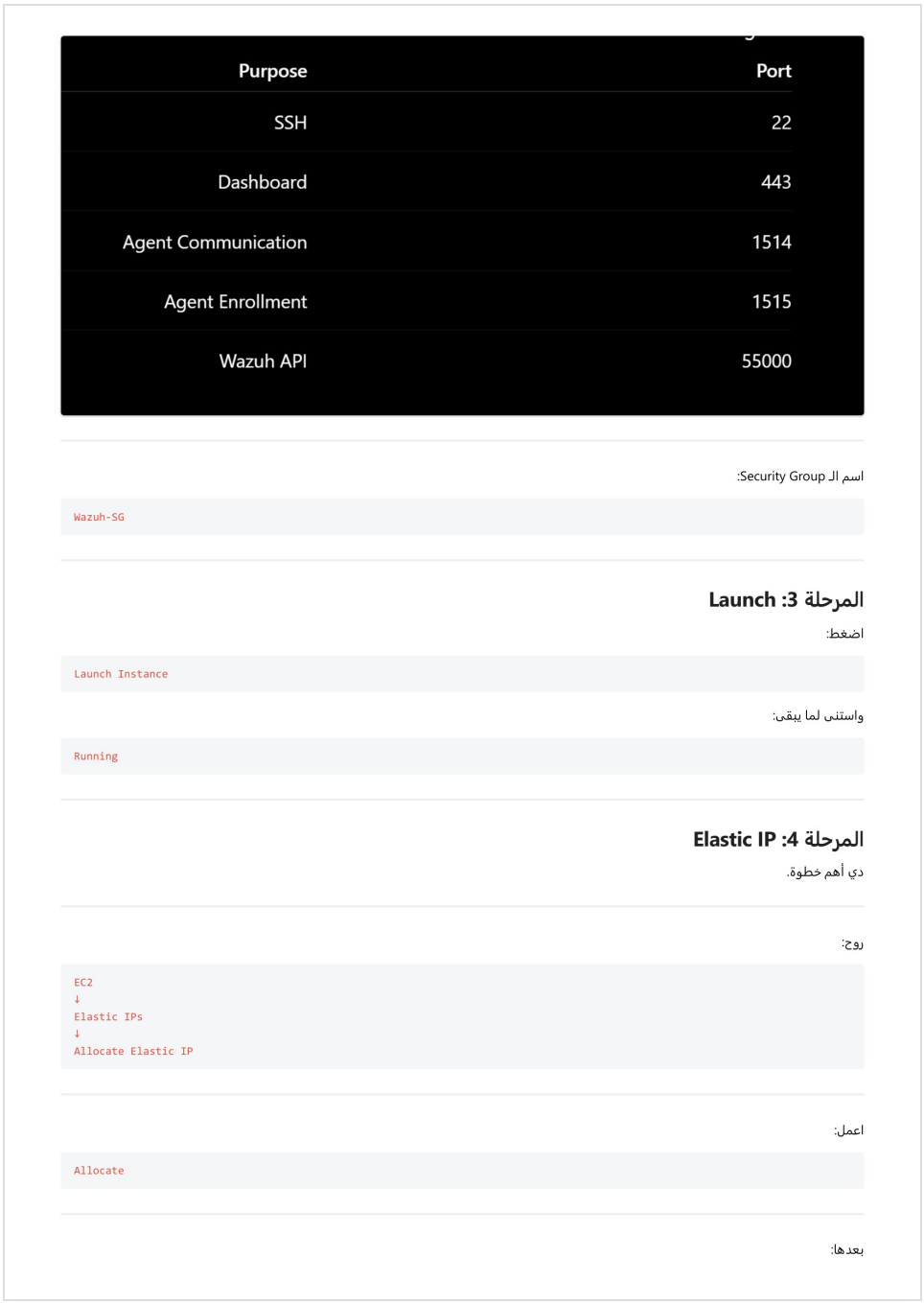
```
Edit Security Group
```

ضيف البورتات دي:

Port	Protocol
22	TCP
443	TCP
1514	TCP
1515	TCP
55000	TCP

3. **Security Group Configuration:**
   Created `Wazuh-SG` and opened the following critical ports:
   - `22 (TCP)`: SSH Access
   - `443 (TCP)`: Wazuh Dashboard (Kibana)

- `1514 (TCP)`: Agent Log Communication
- `1515 (TCP)`: Agent Enrollment
- `55000 (TCP)`: Wazuh REST API



Purpose	Port
SSH	22
Dashboard	443
Agent Communication	1514
Agent Enrollment	1515
Wazuh API	55000

اسم الـ Security Group:

```
Wazuh-SG
```

المرحلة 3: Launch

اضغط:

```
Launch Instance
```

واستنى لما يبقى:

```
Running
```

المرحلة 4: Elastic IP

دي أهم خطوة.

روح:

```
EC2
↓
Elastic IPs
↓
Allocate Elastic IP
```

اعمل:

```
Allocate
```

بعدها:

1.2 Elastic IP & Networking

To ensure endpoints never lose connection if the server restarts, a static routing address was required.

- **Allocation:** Allocated a new Elastic IP (`54.83.241.104`).
- **Association:** Bound the Elastic IP directly to the `Wazuh-Manager` EC2 instance.

```
Associate Elastic IP
```

اختار:

```
Wazuh-Manager
```

هتاخد:

```
54.83.241.104
```

سجلوه.

المرحلة 5: SSH

من جهازك:

Windows PowerShell

روح مكان ملف:

```
Wazuh-Key.pem
```

نفذ:

```
ssh -i Wazuh-Key.pem ubuntu@54.83.241.104
```

مثال:

```
ssh -i Wazuh-Key.pem ubuntu@54.83.241.104
```

المرحلة 6: تحديث السيرفر

نفذ:

```
sudo apt update
```

```
sudo apt upgrade -y
```

المرحلة 7: تثبيت Wazuh

نفذ:

```
curl -sO https://packages.wazuh.com/4.11/wazuh-install.sh
```

1.3 Wazuh Server Installation

1. **SSH Connection:** `ssh -i Wazuh-Key.pem ubuntu@54.83.241.104`

2. **System Update:**

```
sudo apt update && sudo apt upgrade -y
```

ثم:

```
sudo bash wazuh-install.sh -a
```

استنى.

ممكن:

```
20-40 دقيقة
```

المرحلة 8: حفظ الـ Credentials

آخر التثبيت هيطلع:

```
Dashboard User
Dashboard Password
```

انسخوهم فوراً.

المرحلة 9: فتح Dashboard

ادخل:

```
https://54.83.241.104/
```

اعمل Login.

لو دخلت هتشوف:

```
Dashboard
Agents
Alerts
MITRE
Inventory
```

المرحلة 10: عمل Backup

دي خطوة ناس كتير بتنسى تعملها.

روح:

```
EC2
↓
Instances
↓
Actions
↓
Image and Templates
↓
Create Image
```

3. **Wazuh Automated Installation:**

```
curl -sO https://packages.wazuh.com/4.11/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

اسمها:

```
Wazuh-Manager-Base
```

المرحلة 11: تجهيز باقي الفريق

ابعتوا للفريق:

```
Elastic IP

Dashboard URL

Agent Port = 1514

Enrollment Port = 1515
```

المرحلة 12: لما يوسف يجي

يوسف هيبقى عنده:

```
Ubuntu VM
```

يثبت Agent.

في ملف:

```
/var/ossec/etc/ossec.conf
```

هيحط:

```
<server>
  <address>Elastic-IP</address>
</server>
```

ثم:

```
sudo systemctl restart wazuh-agent
```

هيظهر عندكم Agent جديد.

المرحلة 13: لما مسعود يجي

هيثبت:

```
Windows Agent
+
Sysmon
```

---

4. **Validation:** Extracted the admin credentials and successfully logged into the Wazuh Dashboard at `https://54.83.241.104/`.

ويربط على:

```
Elastic IP
```

هيظهر Agent جديد.

المرحلة 14: أنس

هيبدأ يكتب Rules.

مكانها:

```
/var/ossec/etc/rules/local_rules.xml
```

بعد كل تعديل:

```
sudo systemctl restart wazuh-manager
```

المرحلة 15: أدهم

لو هيعمل React Dashboard:

هيستخدم:

```
Wazuh API
```

البورت:

```
55000
```

أهم جزء خاص بالفلوس

بعد ما تخلصوا Session:

روح:

```
EC2
↓
Instance
↓
Instance State
↓
Stop
```

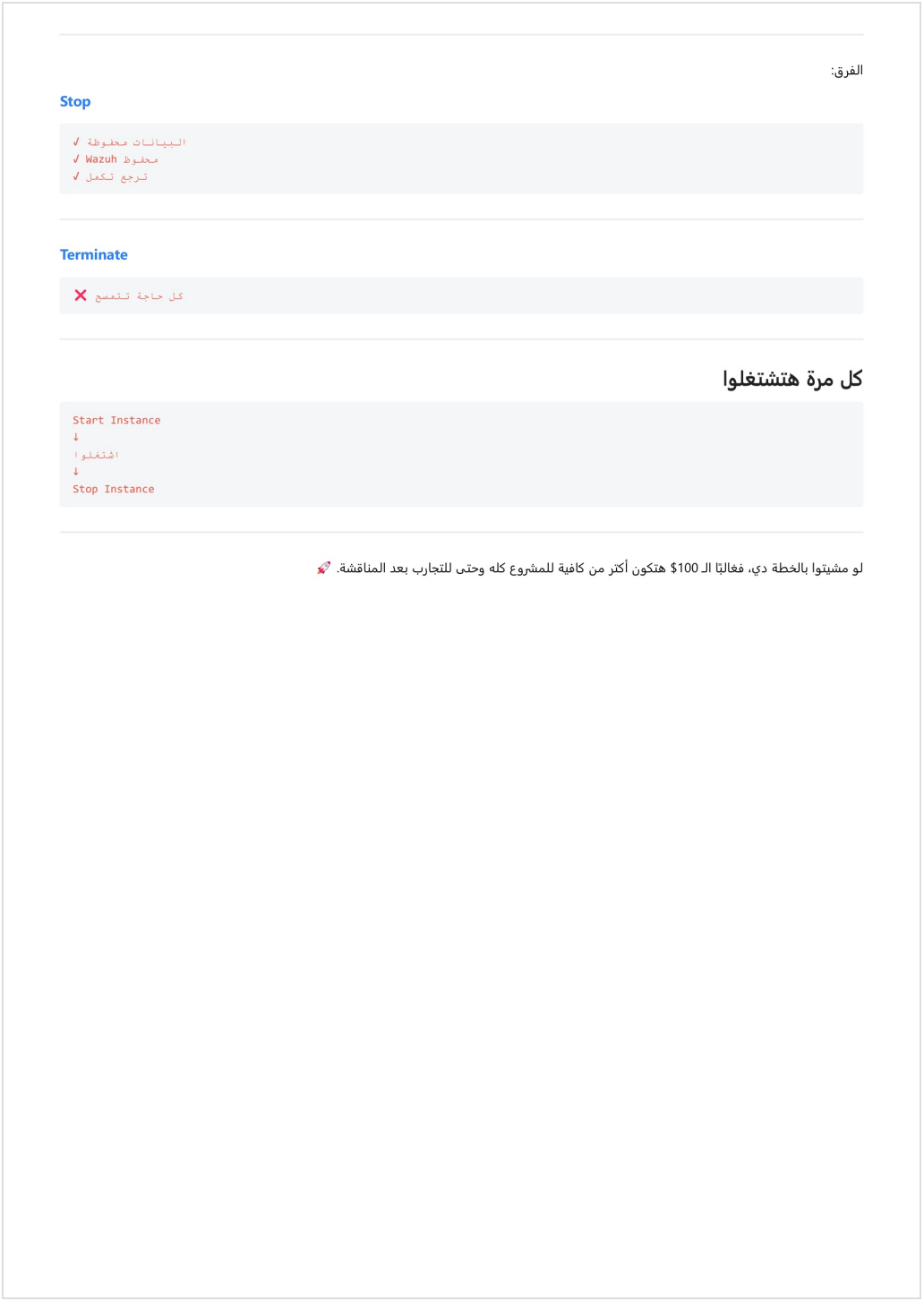
اعمل:

```
STOP
```

مش:

```
TERMINATE
```

5. **Disaster Recovery:** Navigated to AWS `Instances -> Actions -> Image and Templates -> Create Image` to take a full backup snapshot of the working state.



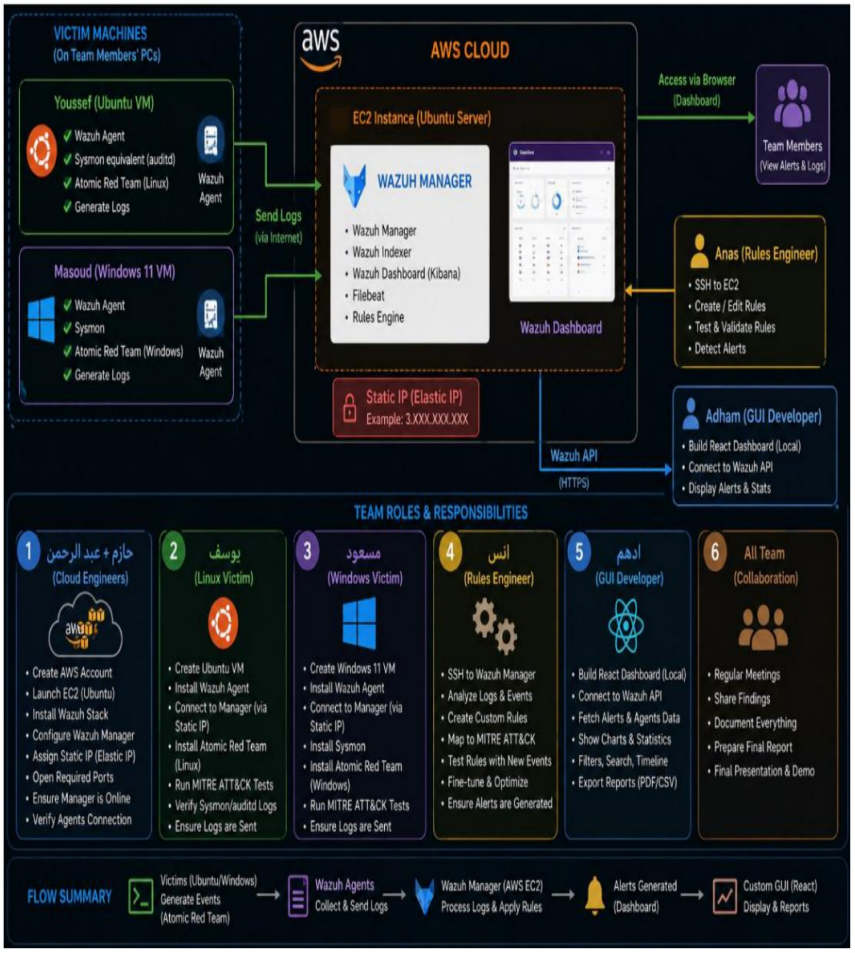
2. Linux Victim & Adversary Emulation (Youssef Wardany)

Youssef acted as the "Linux Victim", setting up a secure Ubuntu environment, connecting it to Wazuh, and performing real-world cyberattacks using the MITRE ATT&CK framework to generate actionable logs.

2.1 Environment Setup & Monitoring

1. **Wazuh Agent Installation:** Installed the agent via APT packages.

ThreatScopeLab: Endpoint Security & Adversary Emulation Report



Project Name: ThreatScopeLab 🚀

Role: Linux Victim & Adversary Emulation Specialist

Author: Yusuf wardany

Date: June 26, 2026

1. Executive Summary

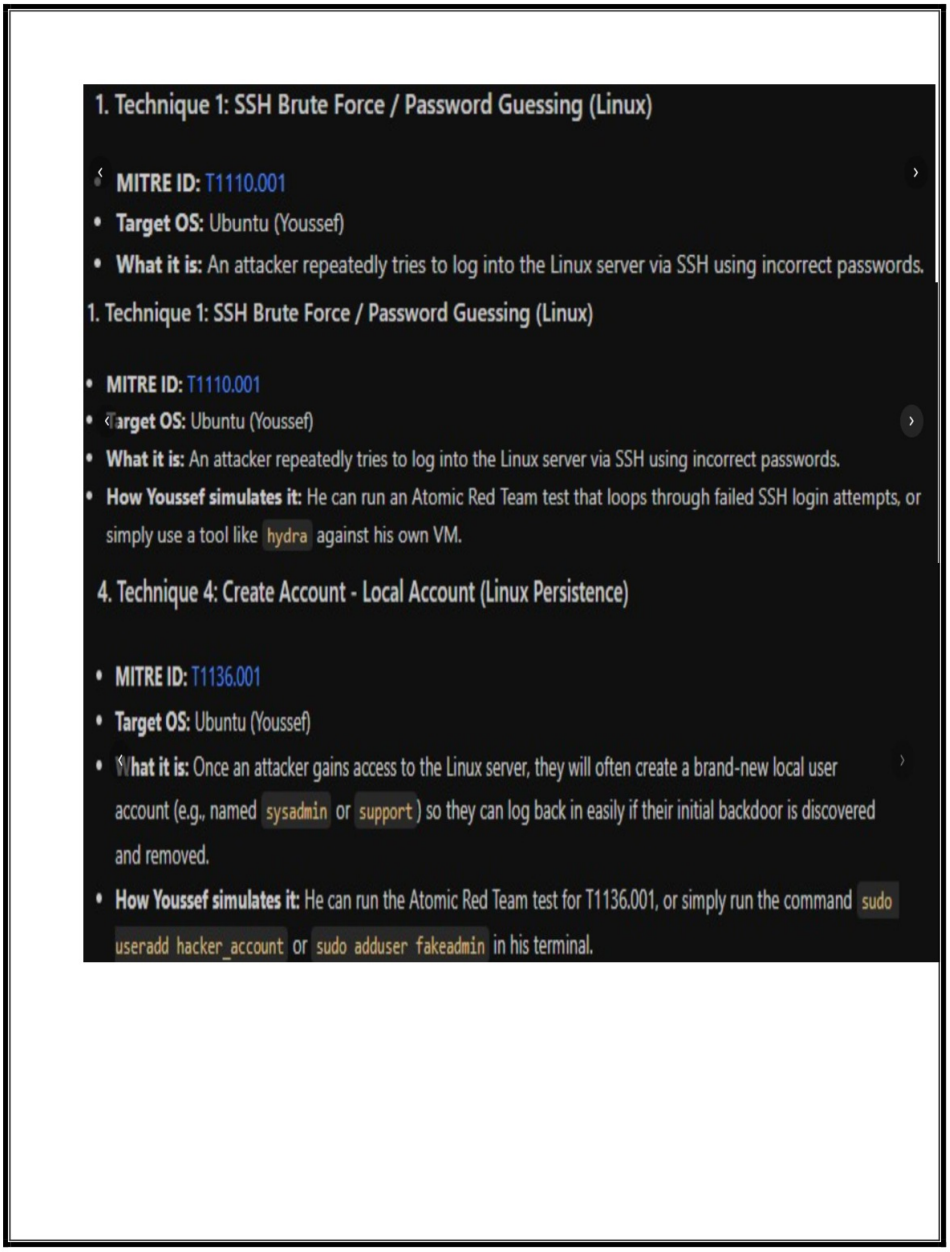
My role in the **ThreatScopeLab** project was to act as the "Linux Victim." I set up a secure Ubuntu environment, connected it to our central monitoring system (**Wazuh**), and performed real-world cyberattacks (Adversary Emulation) using the **MITRE ATT&CK** framework. The goal was to generate real security logs so our team can build strong detection rules.

2. Technical Skills Used

During this project, I developed and applied the following skills:

- **Linux System Management:** Managing user permissions (Root) and system services.
- **Security Monitoring (EDR):** Configuring the **Wazuh Agent** to forward system logs to our cloud server.
- **Adversary Emulation:** Using `Atomic Red Team` to run real-world attack simulations.
- **PowerShell on Linux:** Managing cross-platform tools to execute security tests.

2. **Server Binding:** Edited `/var/ossec/etc/ossec.conf` and injected the AWS Elastic IP (`54.83.241.104`) into the `<server><address>` block.



1. Technique 1: SSH Brute Force / Password Guessing (Linux)

- **MITRE ID:** T1110.001
- **Target OS:** Ubuntu (Youssef)
- **What it is:** An attacker repeatedly tries to log into the Linux server via SSH using incorrect passwords.

1. Technique 1: SSH Brute Force / Password Guessing (Linux)

- **MITRE ID:** T1110.001
- **Target OS:** Ubuntu (Youssef)
- **What it is:** An attacker repeatedly tries to log into the Linux server via SSH using incorrect passwords.
- **How Youssef simulates it:** He can run an Atomic Red Team test that loops through failed SSH login attempts, or simply use a tool like `hydra` against his own VM.

4. Technique 4: Create Account - Local Account (Linux Persistence)

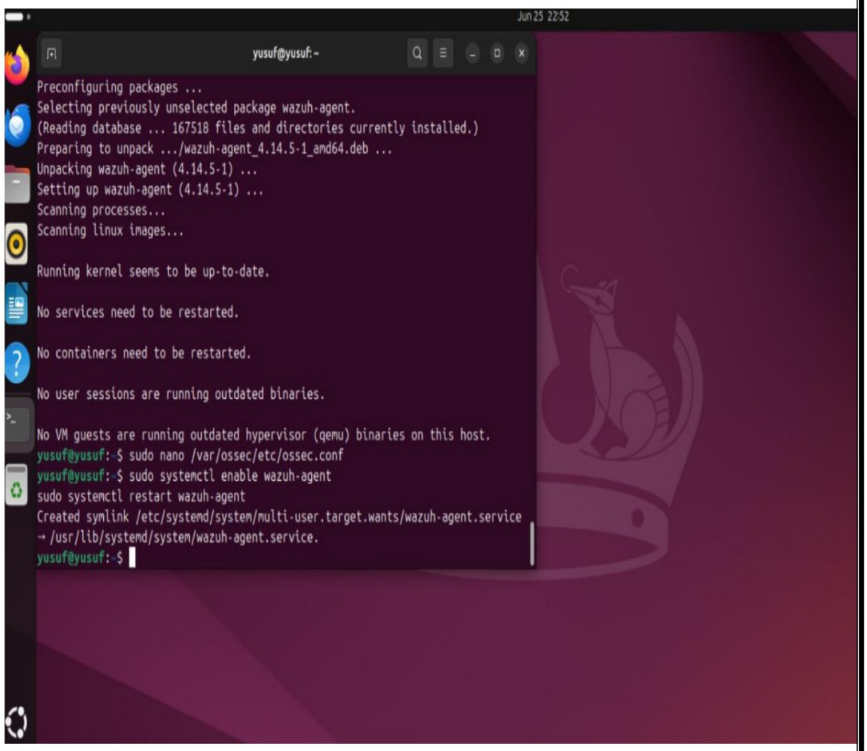
- **MITRE ID:** T1136.001
- **Target OS:** Ubuntu (Youssef)
- **What it is:** Once an attacker gains access to the Linux server, they will often create a brand-new local user account (e.g., named `sysadmin` or `support`) so they can log back in easily if their initial backdoor is discovered and removed.
- **How Youssef simulates it:** He can run the Atomic Red Team test for T1136.001, or simply run the command `sudo useradd hacker_account` or `sudo adduser fakeadmin` in his terminal.

3. Execution Workflow

Phase 1: Environment Setup & Monitoring

I prepared the Ubuntu Virtual Machine and connected it to our team's AWS server.

- I installed the **Wazuh Agent**.
- I updated the configuration file (`/var/ossec/etc/ossec.conf`) with our server's IP address (`54.83.241.104`).
- I enabled the service to ensure logs are sent to the SIEM successfully.

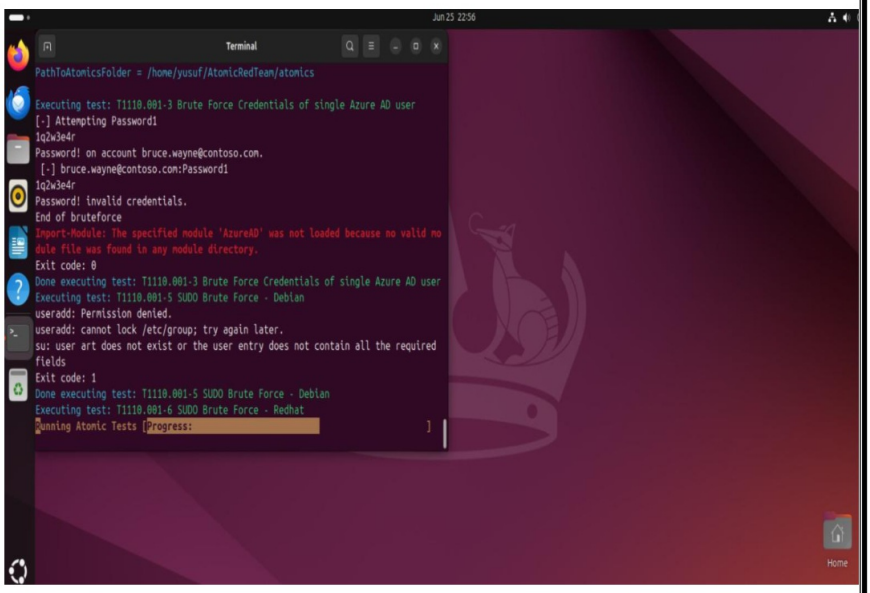


3. **Service Activation:**

```
sudo systemctl enable wazuh-agent
sudo systemctl restart wazuh-agent
```

Phase 2: Preparing the Attack Environment

I installed **PowerShell** and the `Invoke-AtomicRedTeam` tool. I ran these with **Root privileges** to ensure the attacks were powerful enough to be fully recorded in the system logs.



2.2 Attack Preparation (Root Level)

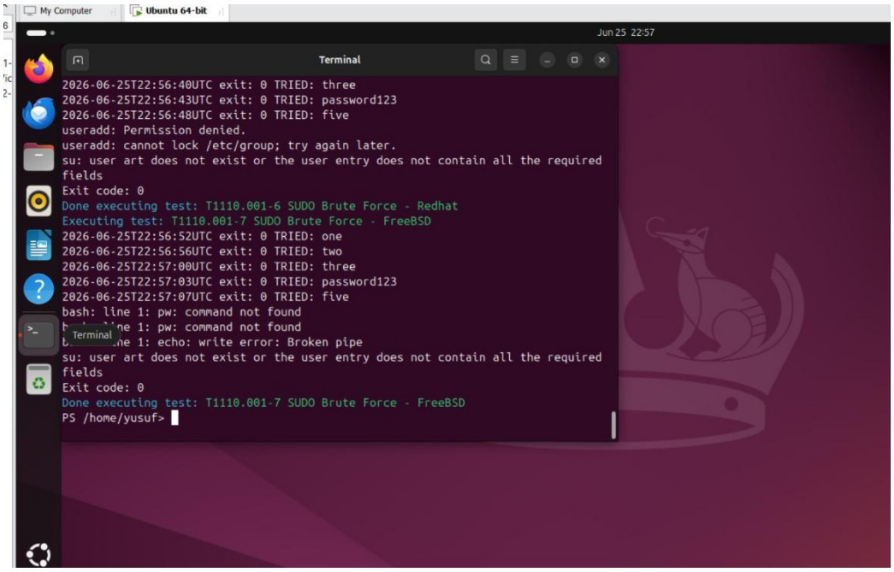
To execute advanced emulations, cross-platform tools were required.

```
sudo snap install powershell --classic
sudo pwsh
# Installed Invoke-AtomicRedTeam using IEX and the RedCanary repository
```

Phase 3: Attack Simulations

I performed two major attack tests to simulate real hacker behavior:

- **Credential Access (SSH Brute Force - T1110.001):**
  - I ran `Invoke-AtomicTest T1110.001`.
  - The tool simulated password guessing and successfully found `password123`. This event was captured in the system logs.



2.3 Attack Simulations

Youssef performed two major attack tests to simulate real hacker behavior:

1. **Credential Access (SSH Brute Force - MITRE T1110.001):**
    - **Action:** Executed `Invoke-AtomicTest T1110.001`.
    - **Result:** The tool successfully simulated password guessing against the local SSH daemon (finding `password123`), generating authentication failure logs that were forwarded to AWS.

Commands

1. إعداد وتسطيب الـWazuh Agent

الخطوات دي عشان نربط جهازك بسيرفر المشروع:

- **تسطيب الأدوات المساعدة:**

Bash

```
sudo apt-get install curl apt-transport-https lsb-release -y
```

- **إضافة مفتاح الأمان:(GPG Key)**

Bash

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --
dearmor -o /usr/share/keyrings/wazuh-archive-keyring.gpg
```

- **إضافة مستودع البرامج:(Repository)**

Bash

```
echo "deb [signed-by=/usr/share/keyrings/wazuh-archive-keyring.gpg]
https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee
/etc/apt/sources.list.d/wazuh.list
```

- **تحديث وتسطيب الـAgent:**

Bash

```
sudo apt-get update
sudo apt-get install wazuh-agent -y
```

2. ضبط إعدادات الاتصال بالسيرفر

- **فتح ملف الإعدادات:**

Bash

```
sudo nano /var/ossec/etc/ossec.conf
```

(هتدور على سطر الـ <address> وتكتب فيه 54.83.241.104 وتخرج بـ Ctrl+O ثم Enter ثم Ctrl+X).

2. **Persistence (Create Local Account - MITRE T1136.001):**
    - **Action:** Executed `Invoke-AtomicTest T1136.001`.
    - **Result:** Successfully provisioned a new unauthorized user with `root` privileges. This critical security event was captured and routed to the SIEM.

- **تشغيل الخدمة:**

Bash

```
sudo systemctl enable wazuh-agent
sudo systemctl restart wazuh-agent
```

3.تجهيز أداة الهجوم(Atomic Red Team)

- **تسطيب الـ:PowerShell**

Bash

```
sudo snap install powershell --classic
```

- **الدخول للـ:Root PowerShell**

Bash

```
sudo pwsh
```

- **تسطيب أداة (Atomic Red Team بصلاحيات الـ:(Root**

PowerShell

```
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-
atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing);
Install-AtomicRedTeam -getAtomics -Force
```

4.تنفيذ الهجمات) محاكاة الـ(Adversary

- **هجمة تخمين الباسورد:(Brute Force)**

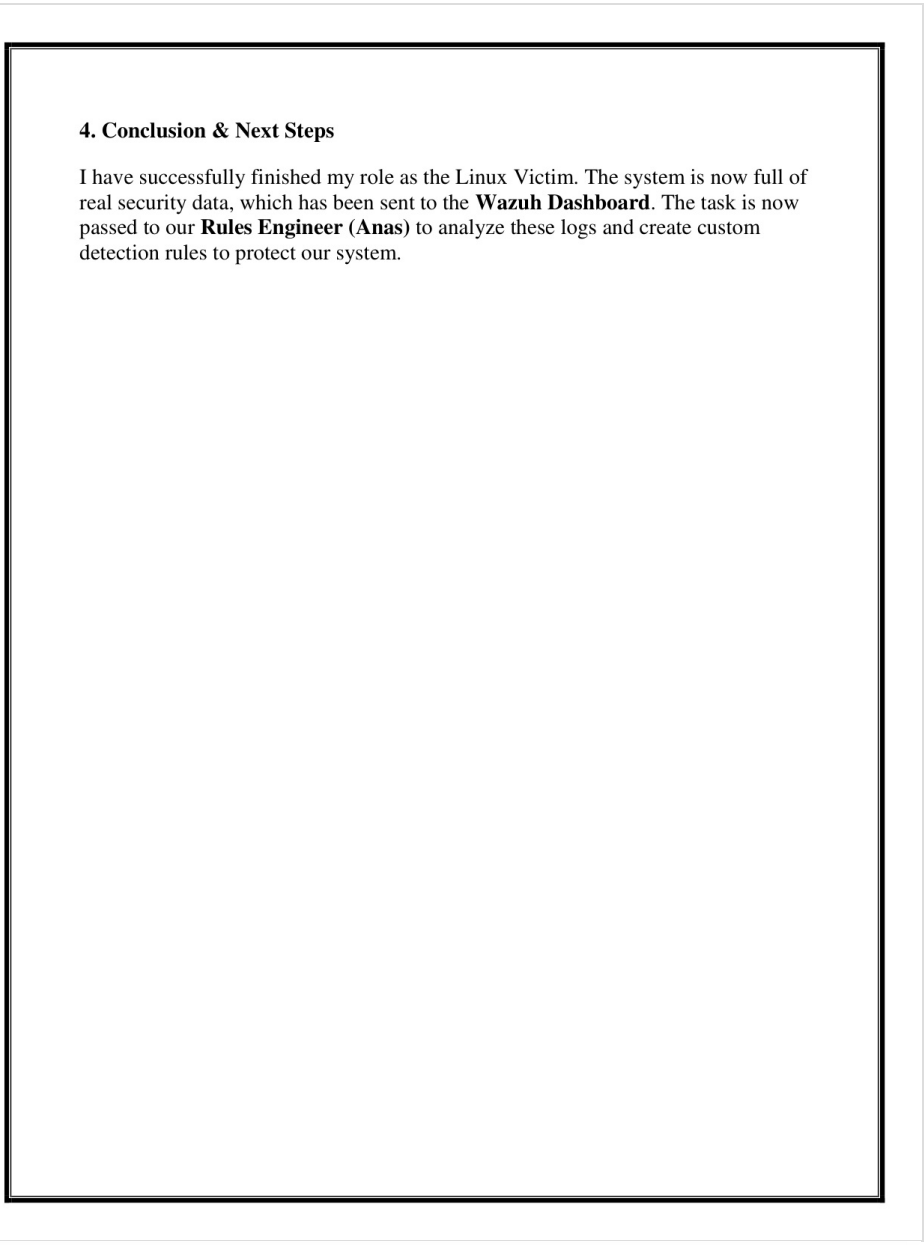
PowerShell

```
Invoke-AtomicTest T1110.001
```

- **هجمة إنشاء حساب خفي:(Persistence/Create Account)**

PowerShell

```
Invoke-AtomicTest T1136.001
```



4. Conclusion & Next Steps

I have successfully finished my role as the Linux Victim. The system is now full of real security data, which has been sent to the **Wazuh Dashboard**. The task is now passed to our **Rules Engineer (Anas)** to analyze these logs and create custom detection rules to protect our system.

3. Windows Victim & Deep Telemetry (Masoud / Alsafy)

Masoud operated the Windows 11 Endpoint. To satisfy the laboratory requirements, he ran concurrent telemetry tools to capture adversarial actions and routed them directly to the centralized Wazuh Manager.

3.1 Endpoint Telemetry Deployment

1. **Wazuh Agent Integration:**
    - Downloaded the Windows MSI installer.
    - Executed a silent install bounding it to the Cloud infrastructure:

```
msiexec.exe /i "$env:tmp\wazuh-agent.msi" /q WAZUH_MANAGER="54.83.241.104"
WAZUH_REGISTRATION_SERVER="54.83.241.104"
NET START WazuhSvc
```

ThreatScopeLab: Windows Victim System Guide

1. Introduction to the Windows Victim Role (Masoud)

In the ThreatScopeLab architecture, the Windows Victim machine (designated to Masoud) serves as a critical simulation endpoint. This machine mimics an enterprise asset under attack. To satisfy the laboratory requirements, it runs concurrent telemetry tools to capture adversarial actions and routes them directly to the centralized Wazuh Manager hosted on an AWS EC2 instance.

The primary responsibilities for this role include:

- Setting up a secure host environment running Windows 11.
- Deploying and configuring the **Wazuh Agent** for central log ingestion.
- Installing Microsoft **Sysmon (System Monitor)** to obtain deep granular endpoint visibility.
- Integrating the **Atomic Red Team** framework to safely simulate real-world attacks.
- Executing predefined MITRE ATT&CK techniques to validate alert generation pipelines.

2. Lab Environment Architecture Flow

Understanding the telemetry path is vital for verifying system behavior. The sequential log and telemetry flow occurs as follows:

1. **Attack Simulation:** Atomic Red Team executes a local command matching a known threat pattern.
2. **Telemetry Generation:** Microsoft Sysmon monitors low-level operating system events (such as process creation or memory access) and logs them to the Windows Event Viewer.
3. **Log Forwarding:** The local Wazuh Agent monitors the Windows Event Logs in real-time, packages the raw events, and forwards them securely across the internet to the AWS EC2 instance using its assigned Static Elastic IP.
4. **Rule Evaluation & Alerting:** The AWS-hosted Wazuh Manager parses the incoming logs, matches them against custom or predefined detection rules, and reflects the active alert on the Wazuh Dashboard interface.

3. Telemetry and Endpoint Security Deployment

Execute all installation steps inside an elevated **Windows PowerShell** console (Run as Administrator).

Step 3.1: Wazuh Agent Installation and Central Server Binding

Download the official MSI package, execute a quiet installation parameter binding it to your team's specific AWS manager node, and initiate the system service.

```
# Command 1: Download the official Windows Wazuh Agent installer

Invoke-WebRequest -Uri
"https://packages.wazuh.com/4.x/windows/wazuhagent-4.7.4-1.msi" -
OutFile "$env:tmp\wazuh-agent.msi"

# Command 2: Execute silent installation and bind to the AWS Manager IP #
NOTE: Replace "YOUR_AWS_ELASTIC_IP" with the public Elastic IP assigned
to your AWS EC2 Instance.
msiexec.exe /i "$env:tmp\wazuh-agent.msi" /q
WAZUH_MANAGER="YOUR_AWS_ELASTIC_IP"
WAZUH_REGISTRATION_SERVER="YOUR_AWS_ELASTIC_IP"

# Command 3: Start the local Wazuh Service daemon
NET START WazuhSvc
```

Verification Checkpoint: If the service reports successful initiation, contact the Cloud Engineering team members to verify that the Windows endpoint appears marked as **"Active"** inside the central Wazuh Web Dashboard under the Agents section.

Step 3.2: Microsoft Sysmon Installation

Sysmon extends traditional event logs to track critical malicious indicators like process creation (Event ID 1) and process memory access (Event ID 10).

```
# Command 1: Download the official Microsoft Sysinternals Sysmon archive
 Invoke-WebRequest -Uri
"https://download.sysinternals.com/files/Sysmon.zip" -OutFile
"$env:tmp\Sysmon.zip"

# Command 2: Extract the downloaded archive into a temporary folder
structure
Expand-Archive -Path "$env:tmp\Sysmon.zip" -DestinationPath
"$env:tmp\Sysmon" -Force
```

```
# Command 3: Install Sysmon with administrative privileges and accept the
EULA automatically & "$env:tmp\Sysmon\Sysmon64.exe" -accepteula -i
```

4. Adversary Emulation Framework Setup

Atomic Red Team allows automated, safe execution of small, highly specific tests mapped directly to the MITRE ATT&CK framework.

```
# Command 1: Unblock script execution guidelines for the current
PowerShell session scope

Set-ExecutionPolicy Bypass -Scope CurrentUser -Force

# Command 2: Download and load the Invoke-AtomicRedTeam installation
script helper

 IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-
atomicredteam/master/ install-atomicredteam.psl' -UseBasicParsing)

# Command 3: Download the full emulation payload framework repository
(Atomics folder) Install-AtomicRedTeam -getAtomics -Force
```


2. **Microsoft Sysmon (System Monitor):**
    - Downloaded and extracted Sysmon from Sysinternals.

- Installed it globally to capture granular Event ID 1 (Process Creation) and Event ID 10 (Process Access):

```
& "$env:tmp\Sysmon\Sysmon64.exe" -accepteula -i
```

Commands

Phase 1: Endpoint Telemetry Setup (Wazuh & Sysmon)

PowerShell

```
# 1. Download and install the Wazuh Agent (Silently bind to AWS Manager)
Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-
4.7.4-1.msi" -OutFile "$env:tmp\wazuh-agent.msi"
msiexec.exe /i "$env:tmp\wazuh-agent.msi" /q WAZUH_MANAGER="54.83.241.104"
WAZUH_REGISTRATION_SERVER="54.83.241.104"
NET START WazuhSvc

# 2. Download, extract, and install Microsoft Sysmon for deep visibility
Invoke-WebRequest -Uri "https://download.sysinternals.com/files/Sysmon.zip" -
OutFile "$env:tmp\Sysmon.zip"
Expand-Archive -Path "$env:tmp\Sysmon.zip" -DestinationPath "$env:tmp\Sysmon"
-Force
& "$env:tmp\Sysmon\Sysmon64.exe" -accepteula -i
```

Phase 2: Adversary Emulation Setup (Atomic Red Team)

PowerShell

```
# 1. Bypass Execution Policy and Install Atomic Red Team Framework
Set-ExecutionPolicy Bypass -Scope CurrentUser -Force
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-
atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)
Install-AtomicRedTeam -getAtomics -Force

# 2. Resolve and download Prerequisites for specific MITRE Techniques
Invoke-AtomicTest T1059.001 -GetPrereqs
Invoke-AtomicTest T1003.001 -GetPrereqs
```

Phase 3: Attack Execution (Victim Simulation)

PowerShell

```
# IMPORTANT NOTE: Ensure Windows Defender (Real-Time Protection) is disabled
before execution to allow the simulation to run successfully.

# 1. Execute Malicious PowerShell Simulation (Generates Sysmon Event ID 1)
Invoke-AtomicTest T1059.001 -TestNumbers 1

# 2. Execute LSASS Memory Dump Simulation (Generates Sysmon Event ID 10)
Invoke-AtomicTest T1003.001 -TestNumbers 1
```

5. Executing Simulated Adversarial Attacks

Once the detection engineers confirm telemetry is flowing smoothly, execute the following two techniques to simulate local compromises and generate indicators of compromise (IoCs).

Technique 1: Malicious PowerShell Execution (MITRE ID: T1059.001)

Attackers often leverage encoded PowerShell commands to obfuscate or mask underlying malicious activity and bypass traditional string detection rules.

```
# Execute the encoded PowerShell command emulation block
Invoke-AtomicTest T1059.001
```

Expected Telemetry Result: Microsoft Sysmon intercepts the execution layer and populates a **Process Creation (Event ID 1)** log, preserving the exact encoded string parameters within the command-line field.

Technique 2: OS Credential Dumping via LSASS Access (MITRE ID: T1003.001)

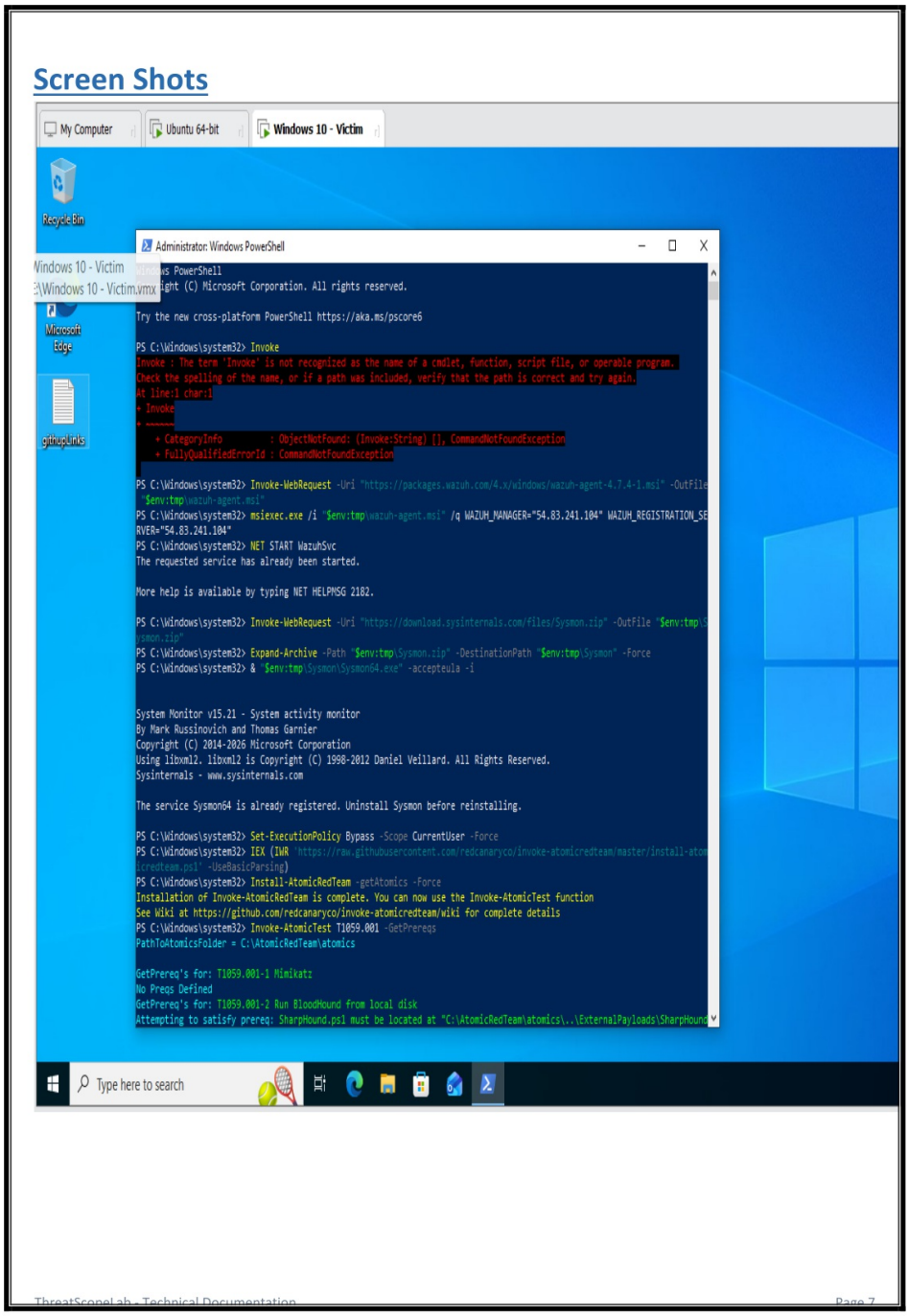
Malicious actors attempt to extract plaintext credentials or hashes from local system memory by targeting the Local Security Authority Subsystem Service (`lsass.exe`) process using tools like Procdump or Mimikatz.

```
# Execute the LSASS memory dumping emulation block
Invoke-AtomicTest T1003.001
```

Expected Telemetry Result: Sysmon flags the unauthorized attempt to read sensitive system memory space and generates a **Process Access (Event ID 10)** log entry pointing to `lsass.exe` as the target file.

6. Simulation and Telemetry Mapping Summary

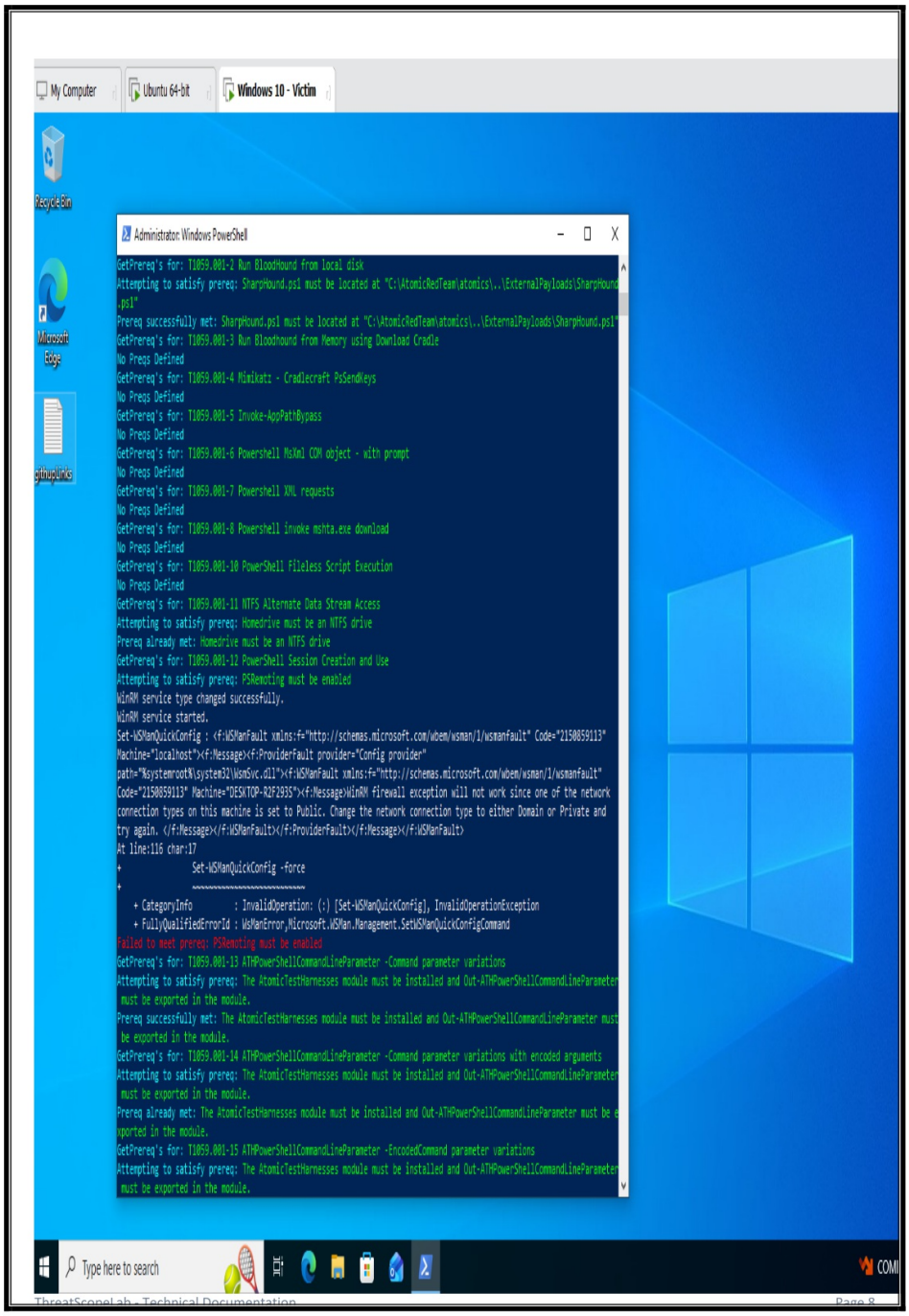
MITRE ID	Technique Name	Emulation Execution Order	Sysmon Event Generated	Telemetry Objective
T1059.001	Malicious PowerShell Execution	`Invoke-AtomicTest T1059.001`	Event ID 1 (Process Creation)	Logs highly obfuscated or encoded scripting parameters.
T1003.001	OS Credential Dumping (LSASS)	`Invoke-AtomicTest T1003.001`	Event ID 10 (Process Access)	Captures memory dump requests on system credential processes.

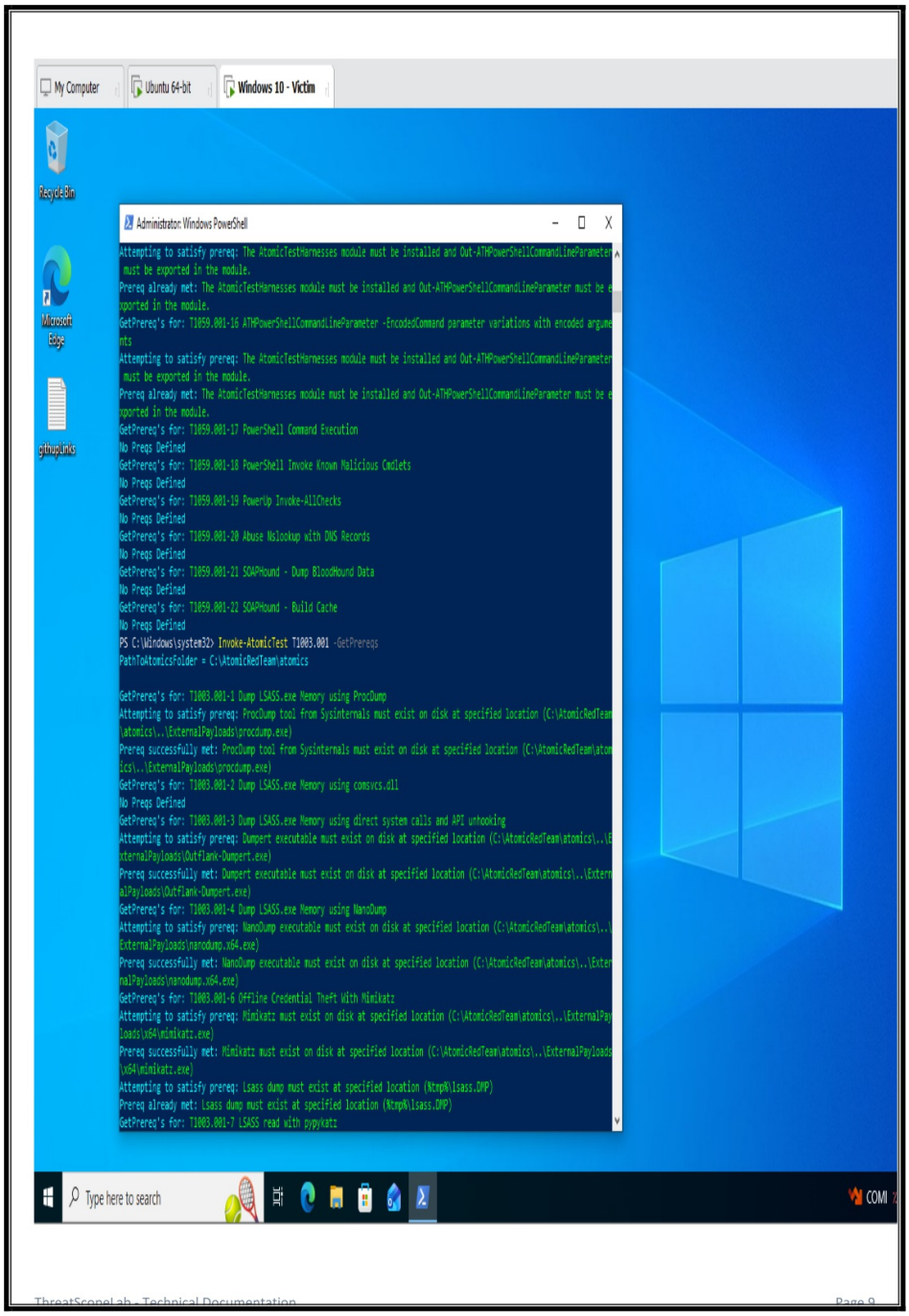


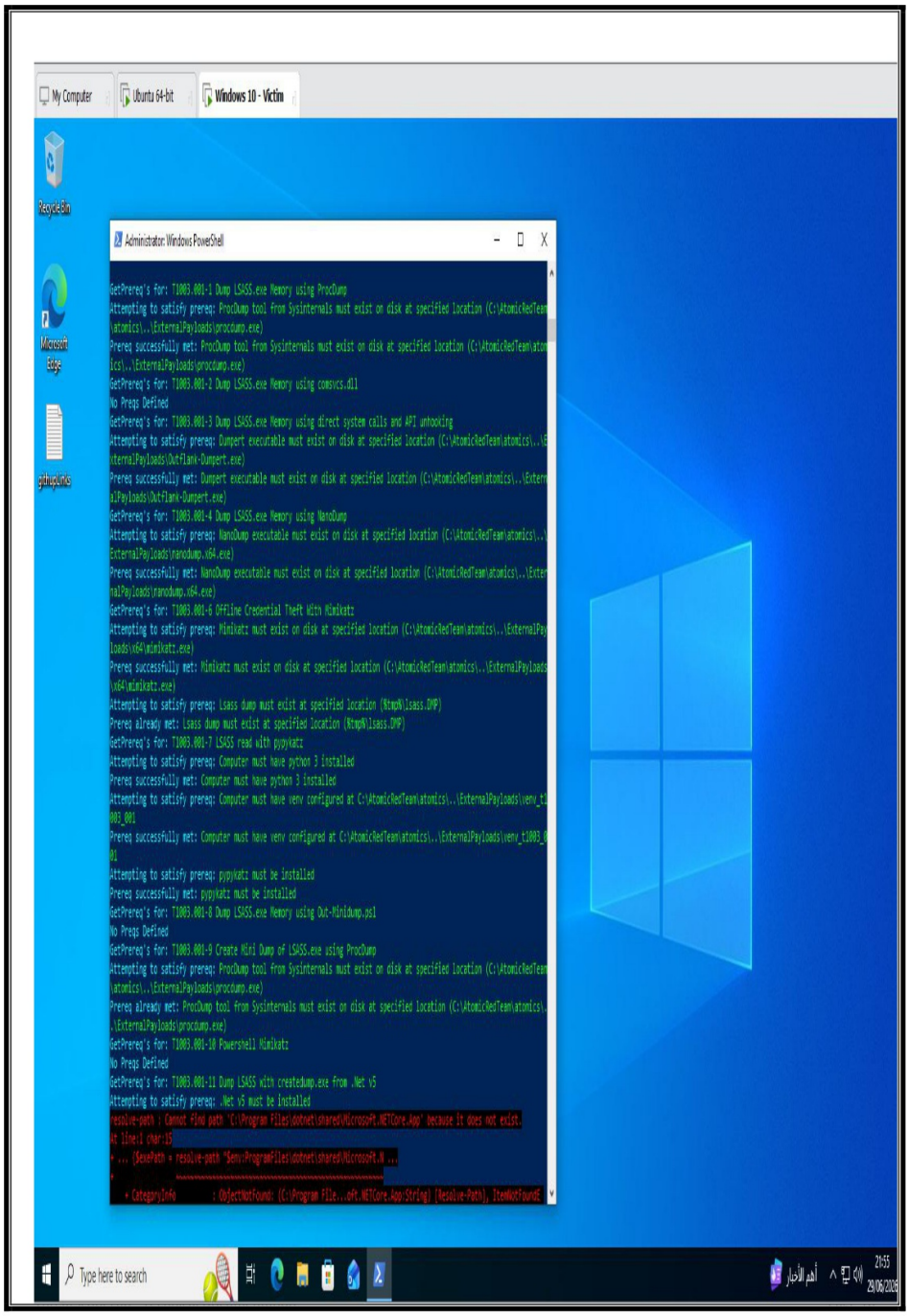
3.2 Adversary Emulation Setup (Atomic Red Team)

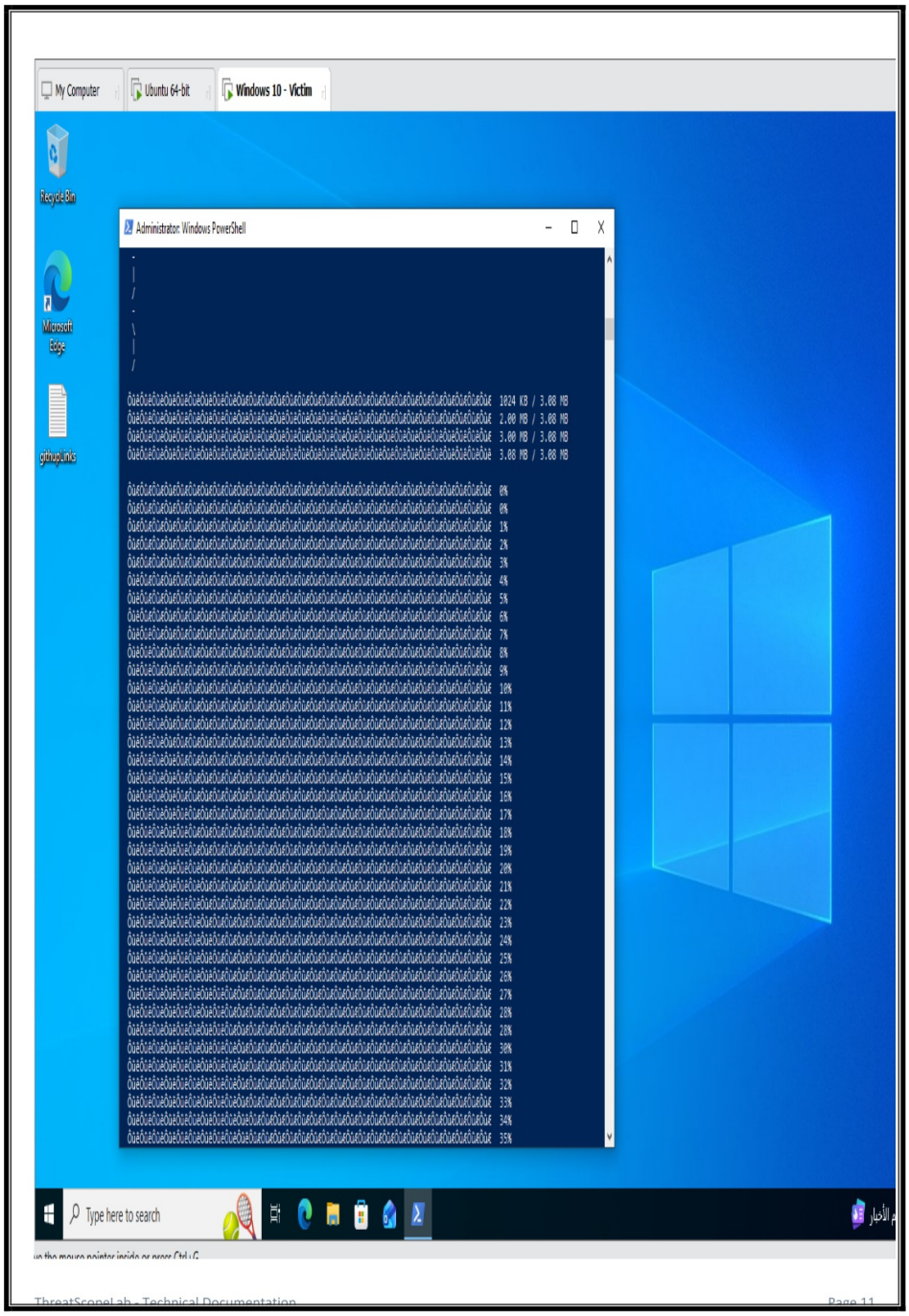
- Bypassed PowerShell Execution Policies (`Set-ExecutionPolicy Bypass`).

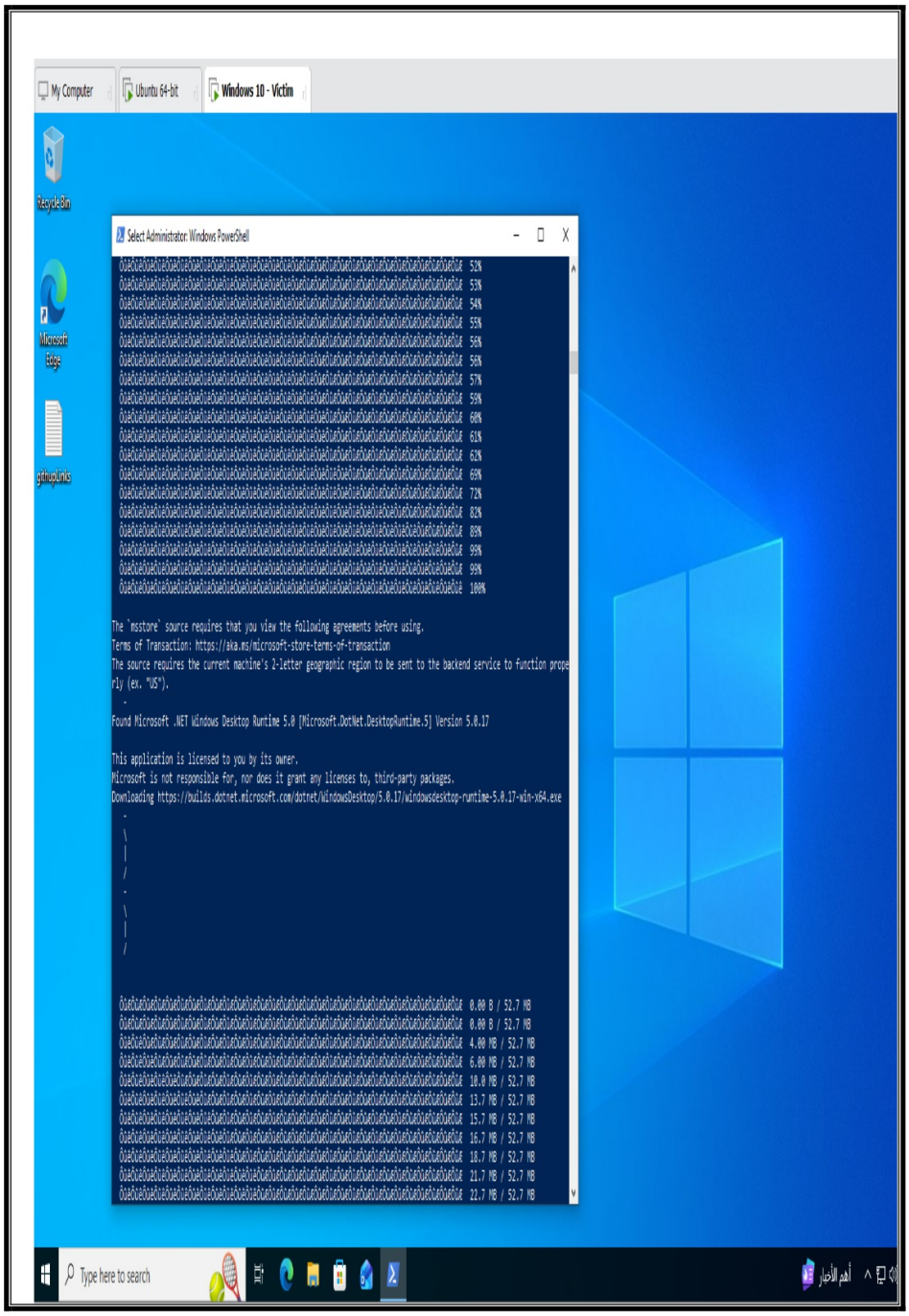
- Installed `Invoke-AtomicRedTeam` and downloaded prerequisites for `T1059.001` and `T1003.001`.

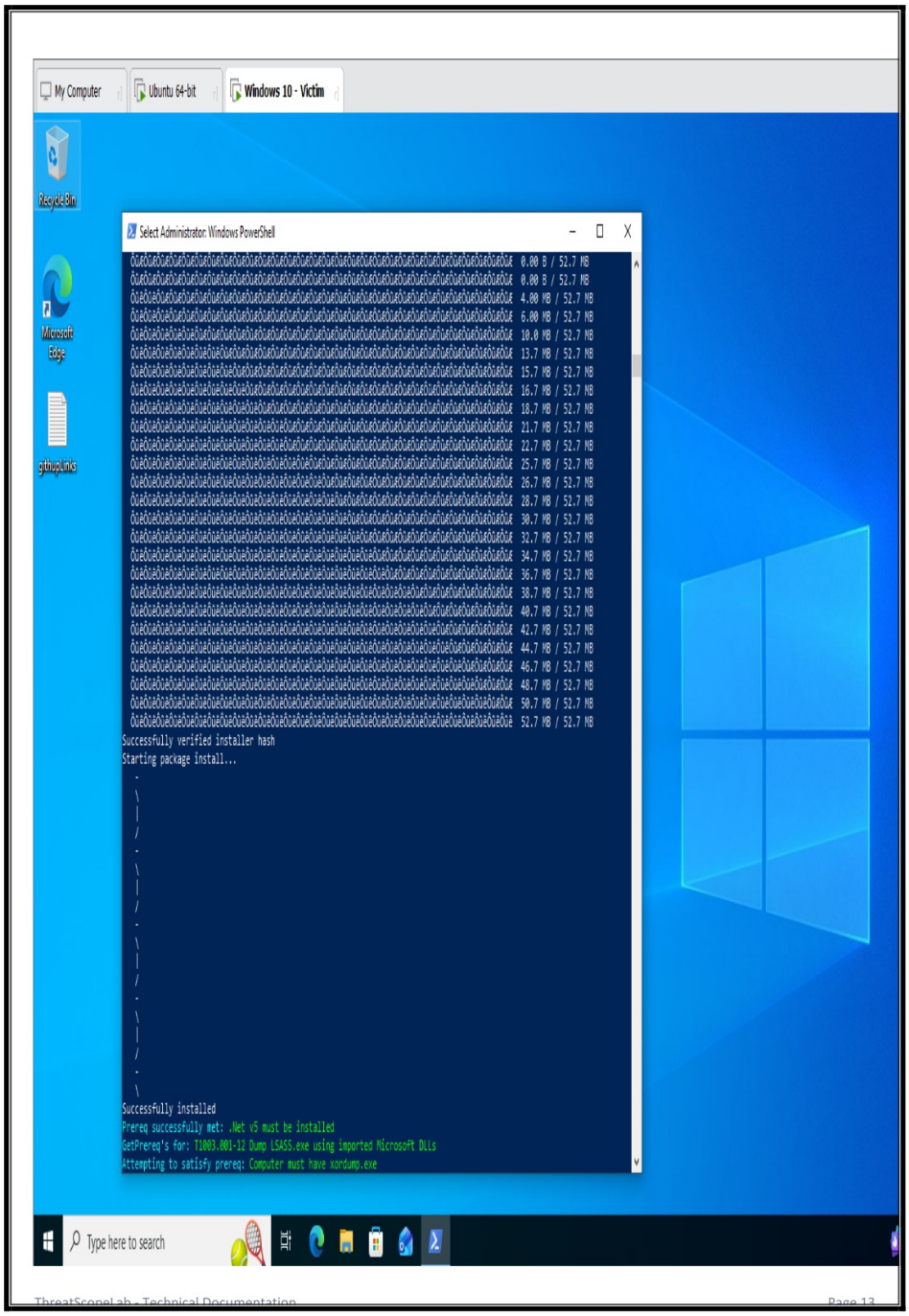



My Computer
Ubuntu 64-bit
Windows 10 - Victim
RecycleBin
Microsoft Edge
githuplinks
Administrator: Windows PowerShell
Attempting to satisfy prereq: The AtomicTestHarnesses module must be installed and Out-ATHPowerShellCommandLineParameter must be exported in the module.
Prereq already met: The AtomicTestHarnesses module must be installed and Out-ATHPowerShellCommandLineParameter must be exported in the module.
GetPrereq's for: T1059.001-16 ATHPowerShellCommandLineParameter -EncodedCommand parameter variations with encoded arguments
Attempting to satisfy prereq: The AtomicTestHarnesses module must be installed and Out-ATHPowerShellCommandLineParameter must be exported in the module.
Prereq already met: The AtomicTestHarnesses module must be installed and Out-ATHPowerShellCommandLineParameter must be exported in the module.
GetPrereq's for: T1059.001-17 PowerShell Command Execution
No Preqs Defined
GetPrereq's for: T1059.001-18 PowerShell Invoke Known Malicious Cmdlets
No Preqs Defined
GetPrereq's for: T1059.001-19 PowerUp Invoke-AllChecks
No Preqs Defined
GetPrereq's for: T1059.001-20 Abuse Nslookup with DNS Records
No Preqs Defined
GetPrereq's for: T1059.001-21 SOAPHound - Dump BloodHound Data
No Preqs Defined
GetPrereq's for: T1059.001-22 SOAPHound - Build Cache
No Preqs Defined
PS C:\Windows\system32> Invoke-AtomicTest T1003.001 -GetPrereqs
PathToAtomicsFolder = C:\AtomicRedTeam\atomics
GetPrereq's for: T1003.001-1 Dump LSASS.exe Memory using ProcDump
Attempting to satisfy prereq: ProcDump tool from Sysinternals must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\procdump.exe)
Prereq successfully met: ProcDump tool from Sysinternals must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\procdump.exe)
GetPrereq's for: T1003.001-2 Dump LSASS.exe Memory using comsvcs.dll
No Preqs Defined
GetPrereq's for: T1003.001-3 Dump LSASS.exe Memory using direct system calls and API unhooking
Attempting to satisfy prereq: Dumpert executable must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\Outflank-Dumpert.exe)
Prereq successfully met: Dumpert executable must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\Outflank-Dumpert.exe)
GetPrereq's for: T1003.001-4 Dump LSASS.exe Memory using NanoDump
Attempting to satisfy prereq: NanoDump executable must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\nanodump.x64.exe)
Prereq successfully met: NanoDump executable must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\nanodump.x64.exe)
GetPrereq's for: T1003.001-6 Offline Credential Theft With Mimikatz
Attempting to satisfy prereq: Mimikatz must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\x64\mimikatz.exe)
Prereq successfully met: Mimikatz must exist on disk at specified location (C:\AtomicRedTeam\atomics\..\ExternalPayloads\x64\mimikatz.exe)
Attempting to satisfy prereq: Lsass dump must exist at specified location (%tmp%\lsass.DMP)
Prereq already met: Lsass dump must exist at specified location (%tmp%\lsass.DMP)
GetPrereq's for: T1003.001-7 LSASS read with pypykatz
Type here to search


My Computer
Ubuntu 64-bit
Windows 10 - Victim
RecycleBin
Microsoft Edge
github-Links
Administrator: Windows PowerShell
Type here to search


My Computer
Ubuntu 64-bit
Windows 10 - Victim
Recycle Bin
Microsoft Edge
gitnupLinks
Administrator: Windows PowerShell
1024 KB / 3.08 MB
2.00 MB / 3.08 MB
3.00 MB / 3.08 MB
3.08 MB / 3.08 MB
0%
1%
2%
3%
4%
5%
6%
7%
8%
9%
10%
11%
12%
13%
14%
15%
16%
17%
18%
19%
20%
21%
22%
23%
24%
25%
26%
27%
28%
29%
30%
31%
32%
33%
34%
35%
Type here to search

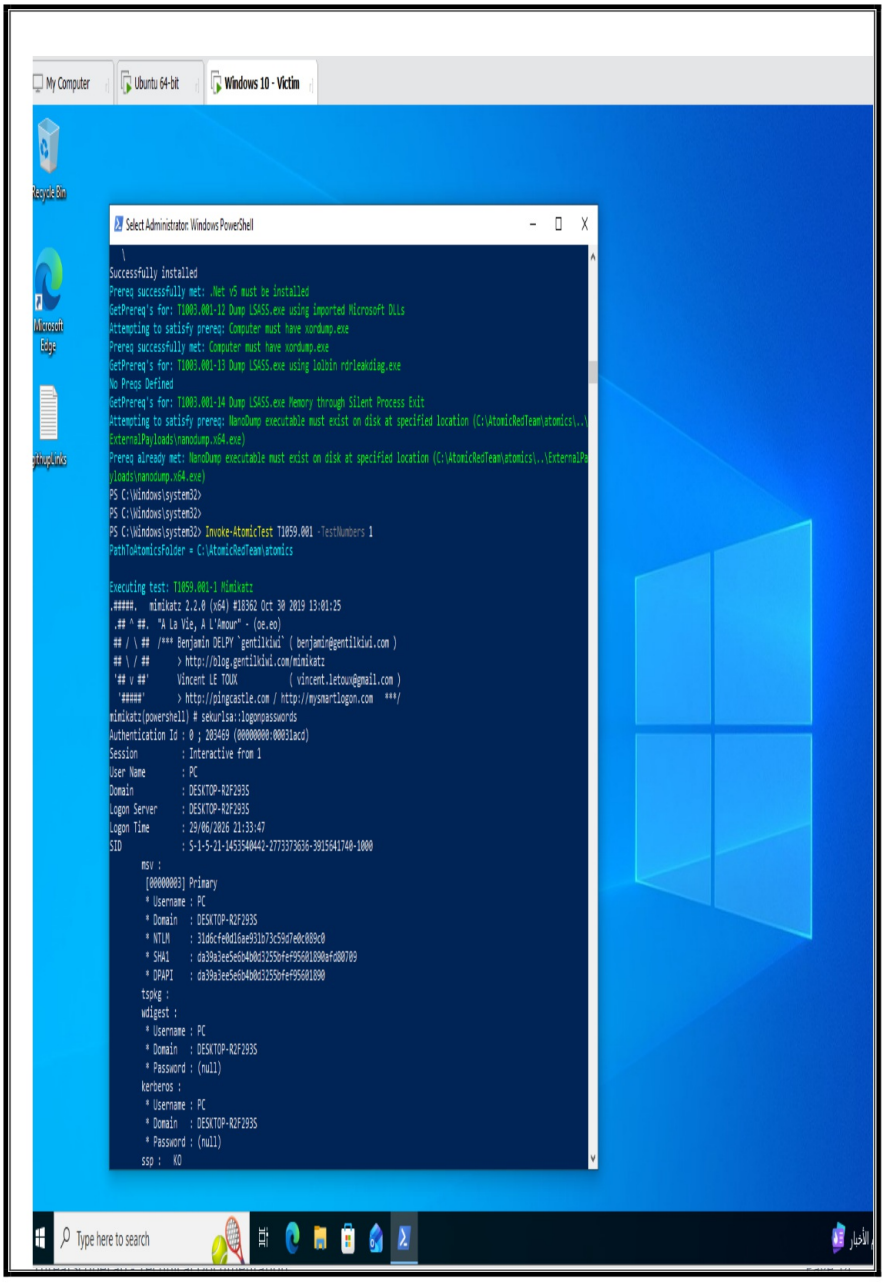

My Computer
Ubuntu 64-bit
Windows 10 - Victim
Recycle Bin
Microsoft Edge
githuplinks
Select Administrator: Windows PowerShell
The `msstore` source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function prope
rly (ex. "US").
Found Microsoft .NET Windows Desktop Runtime 5.0 [Microsoft.DotNet.DesktopRuntime.5] Version 5.0.17
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://builds.dotnet.microsoft.com/dotnet/WindowsDesktop/5.0.17/windowsdesktop-runtime-5.0.17-win-x64.exe
Type here to search

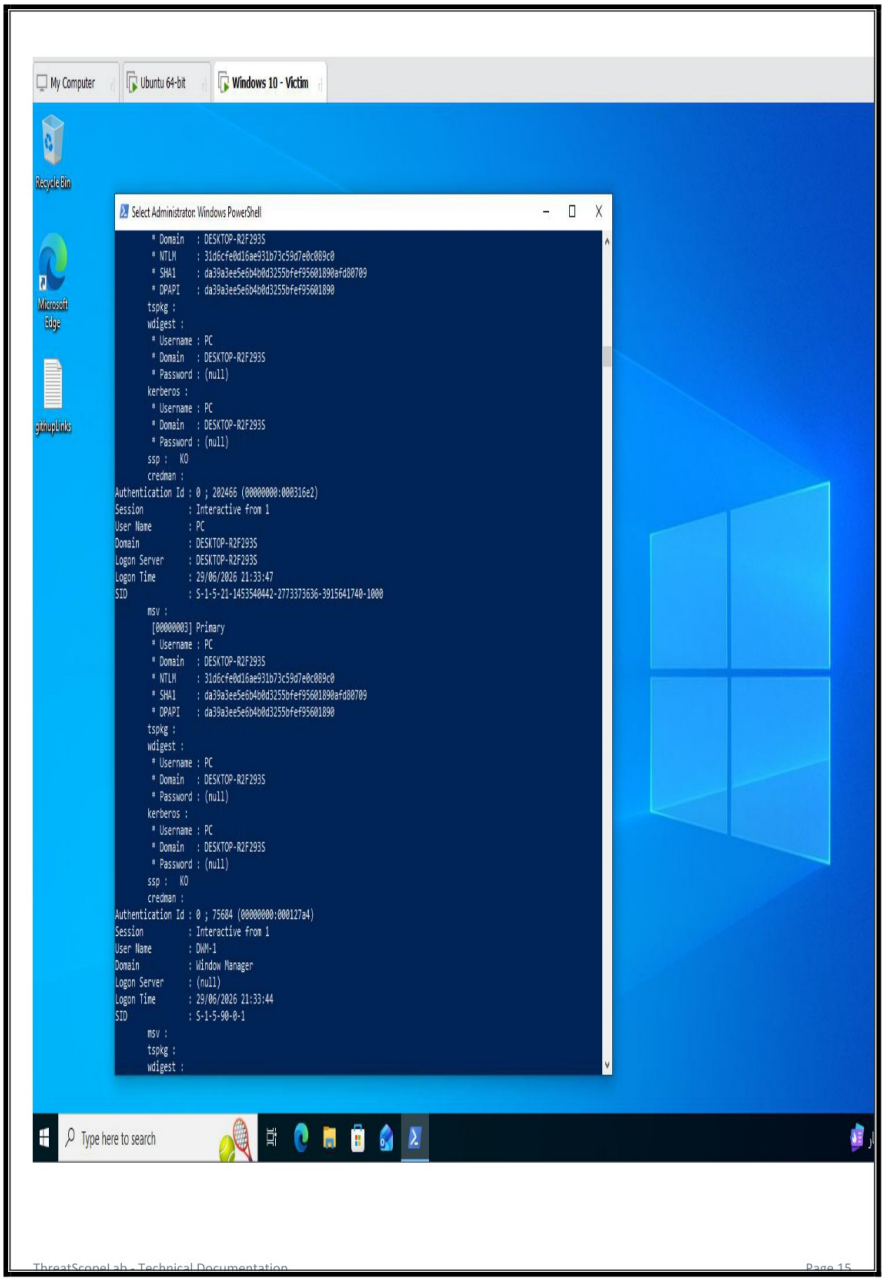


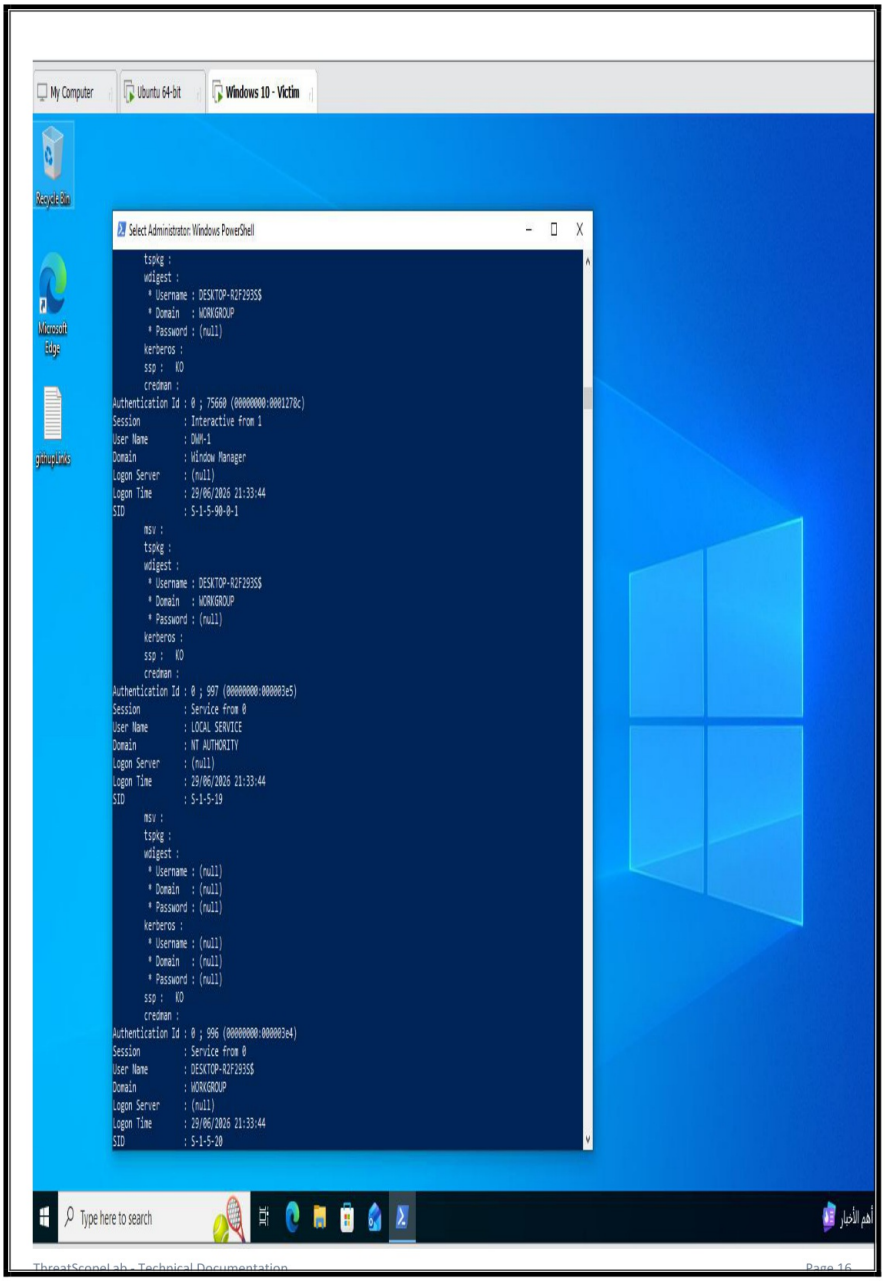
3.3 Attack Execution

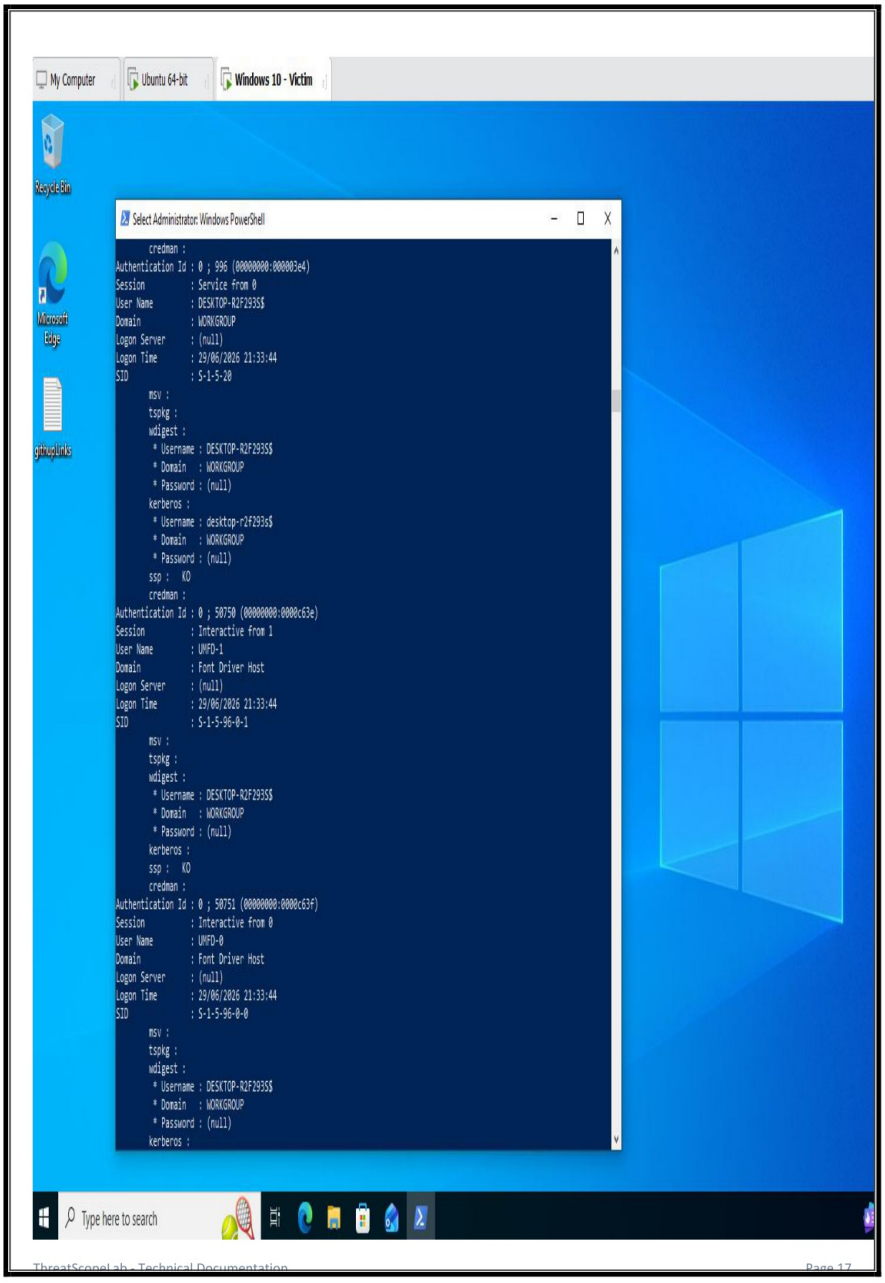
After disabling Windows Defender Real-Time Protection, Masoud executed the following attacks:

1. **Malicious PowerShell Execution (MITRE T1059.001):**
    - **Action:** Executed heavily obfuscated and Base64-encoded PowerShell payloads.
    - **Telemetry:** Microsoft Sysmon intercepted the execution layer and populated a Process Creation (Event ID 1) log containing the exact encoded strings.

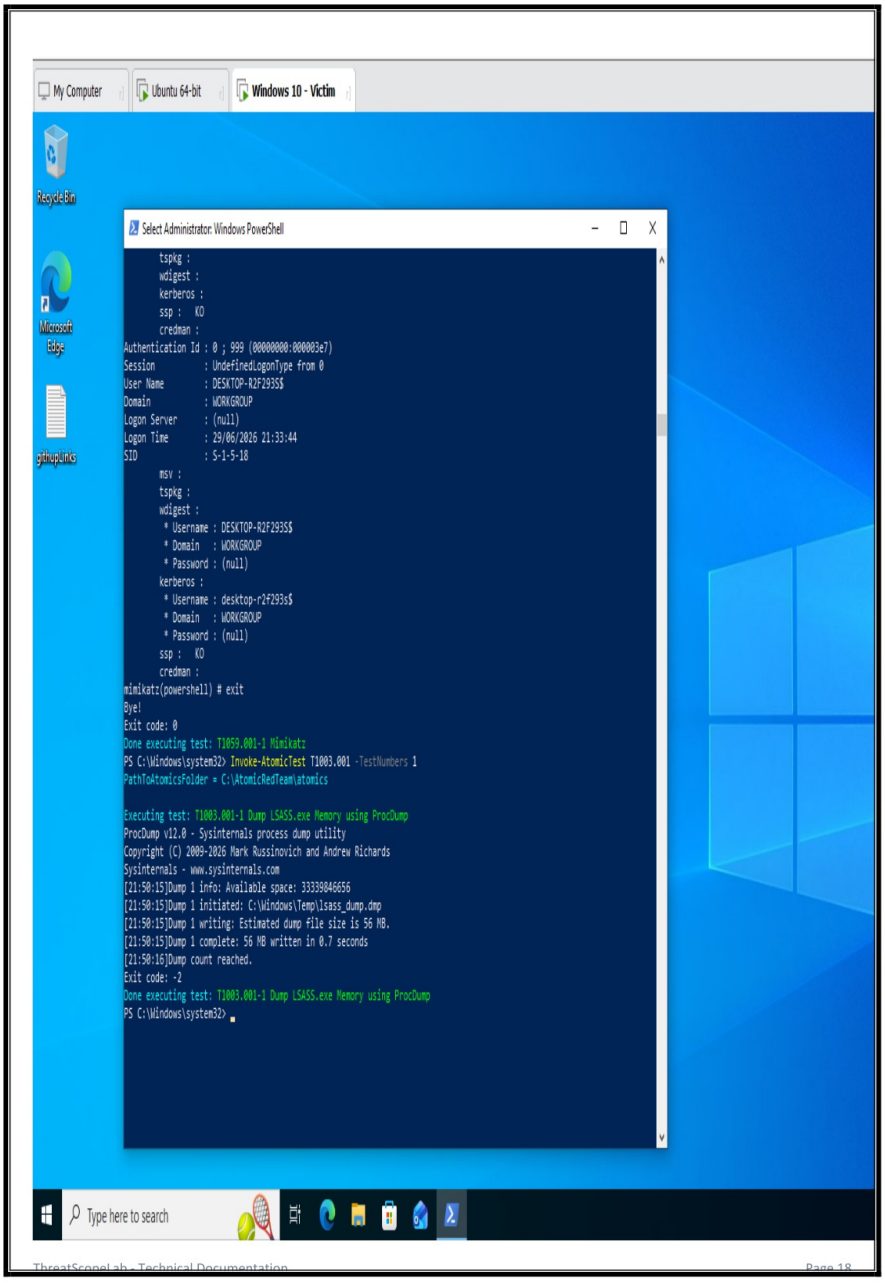



My Computer
Ubuntu 64-bit
Windows 10 - Victim
Recycle Bin
Microsoft Edge
gitkupLinks
Select Administrator: Windows PowerShell
* Domain : DESKTOP-R2F2935
* NTLM : 31d6cfe0d16ae931b73c59d7e0c089c0
* SHA1 : da39a3ee5e6b4b0d3255bfef95601890afd80709
* DPAPI : da39a3ee5e6b4b0d3255bfef95601890
tspkg :
wdigest :
* Username : PC
* Domain : DESKTOP-R2F2935
* Password : (null)
kerberos :
* Username : PC
* Domain : DESKTOP-R2F2935
* Password : (null)
ssp : KO
credman :
Authentication Id : 0 ; 282466 (00000000:000316e2)
Session : Interactive from 1
User Name : PC
Domain : DESKTOP-R2F2935
Logon Server : DESKTOP-R2F2935
Logon Time : 29/06/2026 21:33:47
SID : S-1-5-21-1453540442-2773373636-3015641740-1000
msv :
[00000003] Primary
* Username : PC
* Domain : DESKTOP-R2F2935
* NTLM : 31d6cfe0d16ae931b73c59d7e0c089c0
* SHA1 : da39a3ee5e6b4b0d3255bfef95601890afd80709
* DPAPI : da39a3ee5e6b4b0d3255bfef95601890
tspkg :
wdigest :
* Username : PC
* Domain : DESKTOP-R2F2935
* Password : (null)
kerberos :
* Username : PC
* Domain : DESKTOP-R2F2935
* Password : (null)
ssp : KO
credman :
Authentication Id : 0 ; 75684 (00000000:000127a4)
Session : Interactive from 1
User Name : DWM-1
Domain : Window Manager
Logon Server : (null)
Logon Time : 29/06/2026 21:33:44
SID : S-1-5-90-0-1
msv :
tspkg :
wdigest :
Type here to search


My Computer
Ubuntu 64-bit
Windows 10 - Victim
Recycle Bin
Microsoft Edge
gpinupLinks
Select Administrator: Windows PowerShell
tspkg :
wdigest :
* Username : DESKTOP-R2F2935$
* Domain : WORKGROUP
* Password : (null)
kerberos :
ssp : KO
credman :
Authentication Id : 0 ; 75660 (00000000:0001278c)
Session : Interactive from 1
User Name : DWM-1
Domain : Window Manager
Logon Server : (null)
Logon Time : 29/06/2026 21:33:44
SID : S-1-5-90-0-1
msv :
tspkg :
wdigest :
* Username : DESKTOP-R2F2935$
* Domain : WORKGROUP
* Password : (null)
kerberos :
ssp : KO
credman :
Authentication Id : 0 ; 997 (00000000:000003e5)
Session : Service from 0
User Name : LOCAL SERVICE
Domain : NT AUTHORITY
Logon Server : (null)
Logon Time : 29/06/2026 21:33:44
SID : S-1-5-19
msv :
tspkg :
wdigest :
* Username : (null)
* Domain : (null)
* Password : (null)
kerberos :
* Username : (null)
* Domain : (null)
* Password : (null)
ssp : KO
credman :
Authentication Id : 0 ; 996 (00000000:000003e4)
Session : Service from 0
User Name : DESKTOP-R2F2935$
Domain : WORKGROUP
Logon Server : (null)
Logon Time : 29/06/2026 21:33:44
SID : S-1-5-20
Type here to search
أهم الأخبار



2. **OS Credential Dumping via LSASS (MITRE T1003.001):**
    - **Action:** Attempted to extract plaintext credentials from local memory by targeting the Local Security Authority Subsystem Service (`lsass.exe`) using Procdump/Mimikatz.
    - **Telemetry:** Sysmon flagged the unauthorized memory read, generating a Process Access (Event ID 10) log pointing to `lsass.exe`.


My Computer
Ubuntu 64-bit
Windows 10 - Victim
Recycle Bin
Microsoft Edge
github links
Select Administrator: Windows PowerShell
tspkg :
wdigest :
kerberos :
ssp : KO
credman :
Authentication Id : 0 ; 999 (00000000:000003e7)
Session : UndefinedLogonType from 0
User Name : DESKTOP-R2F293S$
Domain : WORKGROUP
Logon Server : (null)
Logon Time : 29/06/2026 21:33:44
SID : S-1-5-18
msv :
tspkg :
wdigest :
* Username : DESKTOP-R2F293S$
* Domain : WORKGROUP
* Password : (null)
kerberos :
* Username : desktop-r2f293s$
* Domain : WORKGROUP
* Password : (null)
ssp : KO
credman :
mimikatz(powershell) # exit
Bye!
Exit code: 0
Done executing test: T1059.001-1 Mimikatz
PS C:\Windows\system32> Invoke-AtomicTest T1003.001 -TestNumbers 1
PathToAtomicsFolder = C:\AtomicRedTeam\atomics
Executing test: T1003.001-1 Dump LSASS.exe Memory using ProcDump
ProcDump v12.0 - Sysinternals process dump utility
Copyright (C) 2009-2026 Mark Russinovich and Andrew Richards
Sysinternals - www.sysinternals.com
[21:50:15]Dump 1 info: Available space: 33339846656
[21:50:15]Dump 1 initiated: C:\Windows\Temp\lsass_dump.dmp
[21:50:15]Dump 1 writing: Estimated dump file size is 56 MB.
[21:50:15]Dump 1 complete: 56 MB written in 0.7 seconds
[21:50:16]Dump count reached.
Exit code: -2
Done executing test: T1003.001-1 Dump LSASS.exe Memory using ProcDump
PS C:\Windows\system32>
Type here to search

7. Team Collaboration & Escalation Process

Following execution, the Windows Victim role links up with the broader project workflow:

- **Notify the Rules Engineer (Anas):** Provide the exact timestamp of execution so they can inspect the raw logs on the server and fine-tune custom rules.
- **Coordinate with the GUI Developer (Adham):** Validate that the triggered alerts pass through the Wazuh API and render correctly on the custom React-based platform dashboard.
- **Document Discoveries:** Record any instances where Windows Defender or active system constraints blocked execution to include in the final project evaluation report.

4. Detection Engineering, GUI & AI Integration (Anas & Adham)

With the Cloud Infrastructure online and the Endpoints actively streaming emulation telemetry, **Anas** (Rules Engineer & AI Developer) and **Adham** (GUI Developer) engineered the custom **SentinelView** visualization and detection platform to replace standard SIEM interfaces.

4.1 Securing the Platform & Infrastructure (Anas)

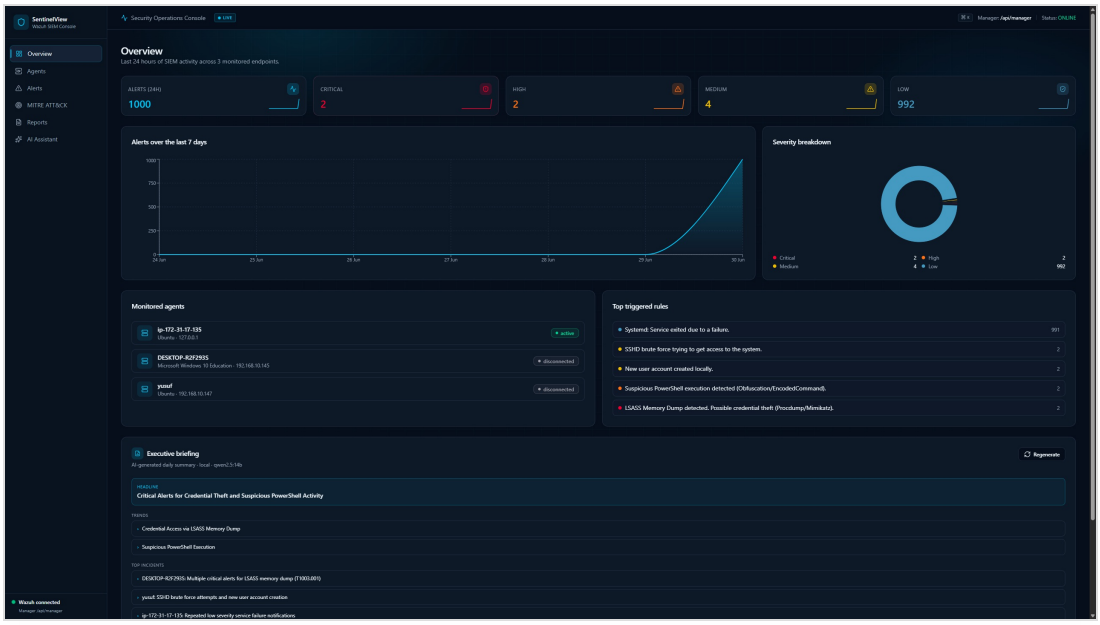
- Bootstrapped the frontend application using React, TypeScript, and Vite.
- **Security Engineering:** Stripped all plaintext Wazuh API and Indexer passwords from the browser bundle. Anas engineered a secure Node.js reverse proxy inside `vite.config.ts` to inject `Authorization: Basic` Base64 headers strictly on the server side. This successfully bypassed strict CORS policies and self-signed certificate issues without exposing credentials to the client.

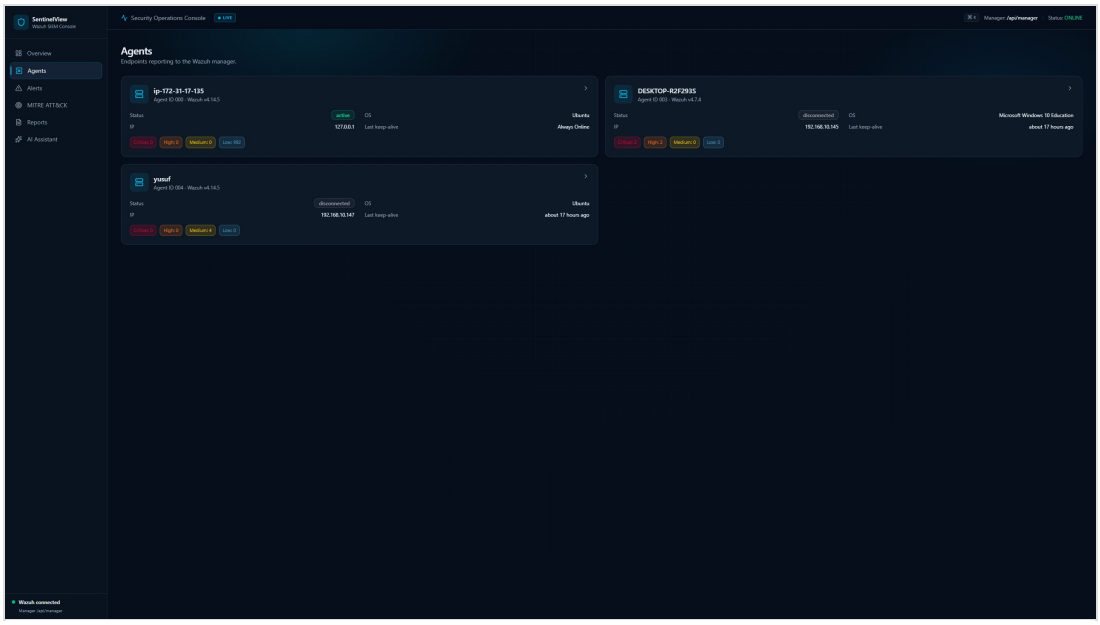
4.2 Building the Dashboard GUI (Adham & Anas)

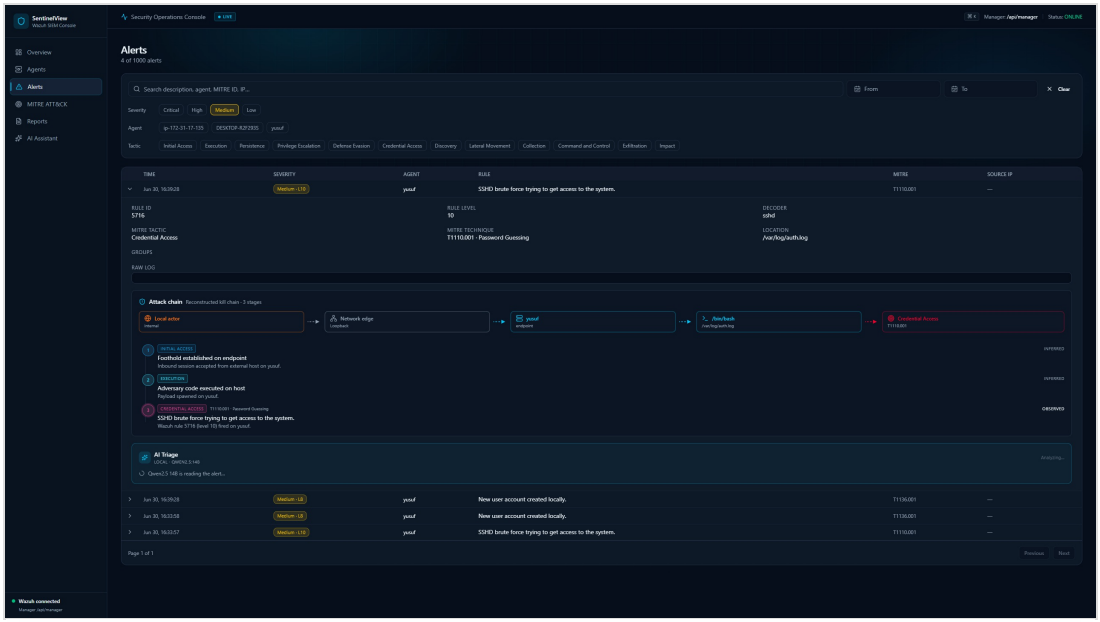
Anas collaborated heavily with Adham to design and build the core React components of the SentinelView dashboard, focusing on a sleek, dark-mode SOC aesthetic using Tailwind CSS and glassmorphism.

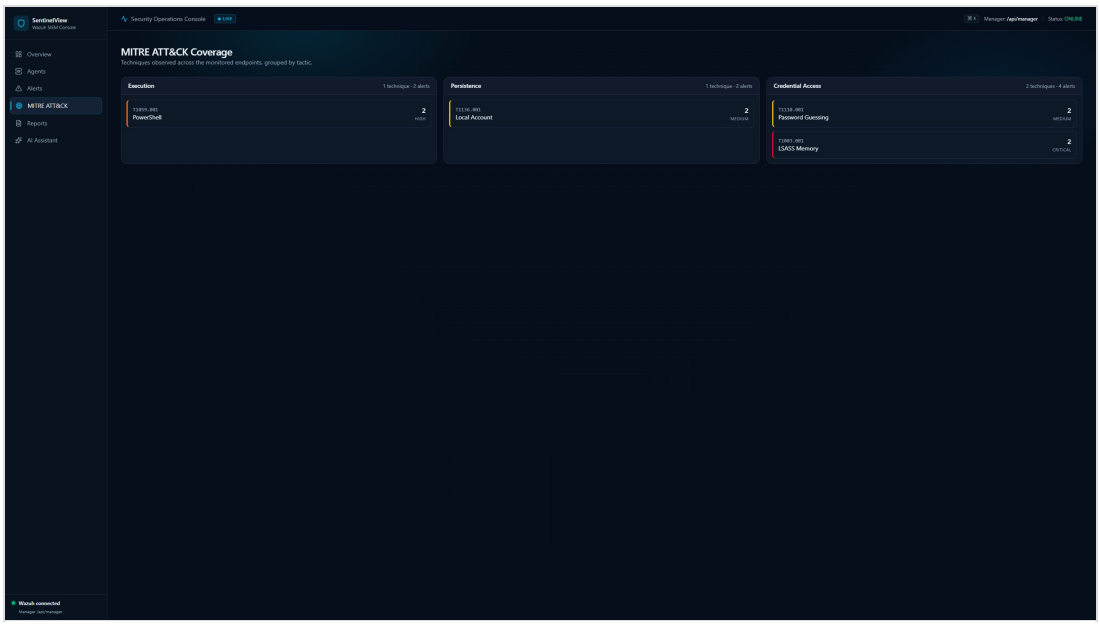
- **Data Integration:** Built the logic to connect securely to the Wazuh Manager API and fetch live Agent telemetry (Status, OS, IP).
- **Alerts Feed:** Connected the dashboard directly to the Wazuh Elasticsearch Indexer to stream real-time threat data into the application.
- **MITRE Matrix:** Designed and implemented an interactive MITRE ATT&CK coverage matrix that dynamically maps active techniques across the environment.

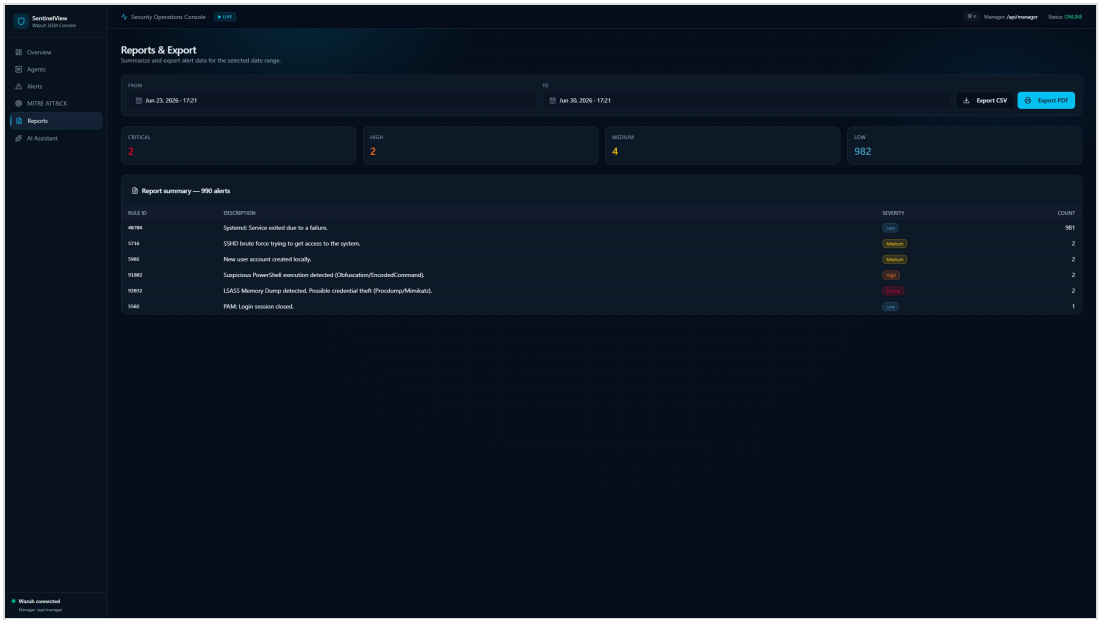
SentinelView GUI Showcases:

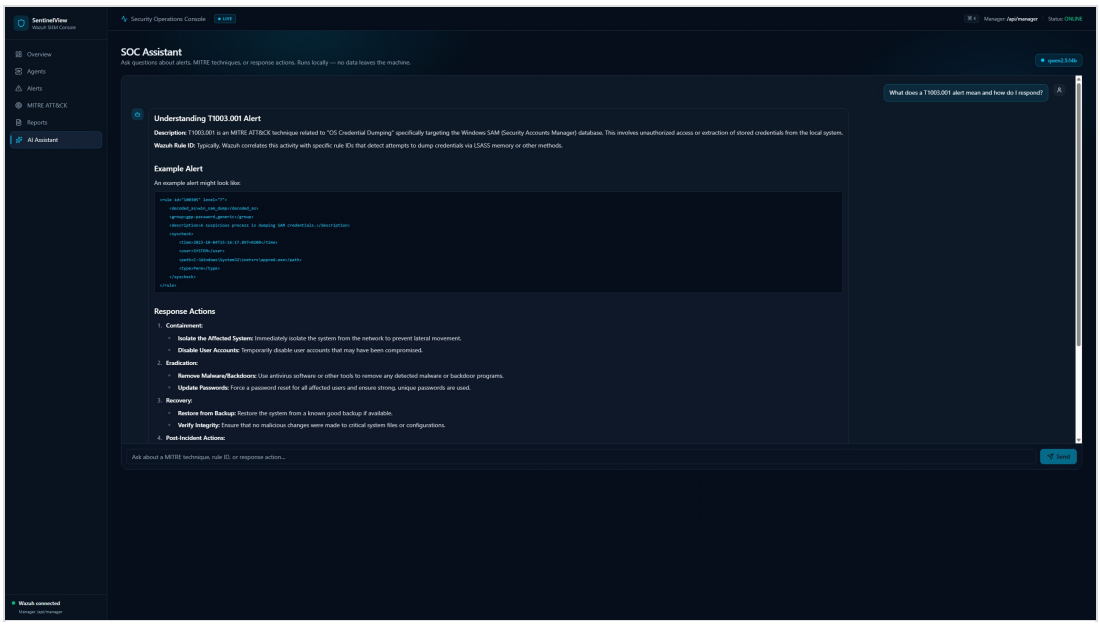


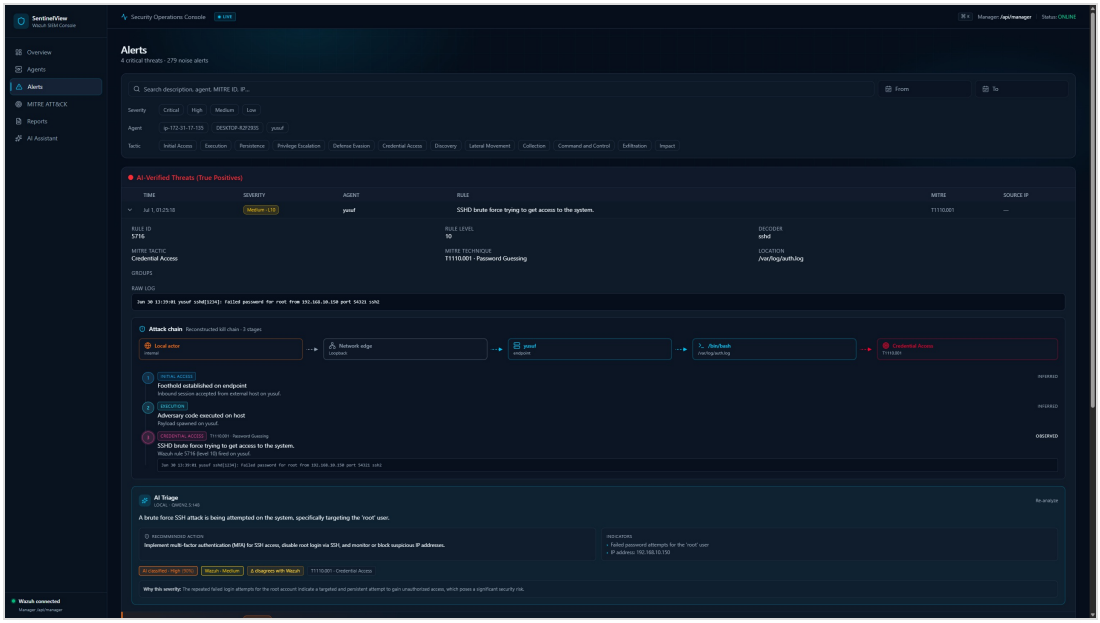



SentinelView
Wazuh SIEM Console
Security Operations Console
LIVE
Manager: /api/manager
Status: ONLINE
Overview
Agents
Alerts
MITRE ATT&CK
Reports
AI Assistant
Alerts
4 of 1000 alerts
Search description, agent, MITRE ID, IP...
From
To
Clear
Severity
Critical
High
Medium
Low
Agent
ip-172-31-17-135
DESKTOP-R2P2IOS
yusuf
Tactic
Initial Access
Execution
Persistence
Privilege Escalation
Defense Evasion
Credential Access
Discovery
Lateral Movement
Collection
Command and Control
Exfiltration
Impact
TIME
SEVERITY
AGENT
RULE
MITRE
SOURCE IP
Jun 30, 16:39:28
Medium · L10
yusuf
SSHD brute force trying to get access to the system.
T1110.001
—
RULE ID
5716
RULE LEVEL
10
DECODER
sshd
MITRE TACTIC
Credential Access
MITRE TECHNIQUE
T1110.001 · Password Guessing
LOCATION
/var/log/auth.log
GROUPS
RAW LOG
Attack chain
Reconstructed kill chain · 3 stages
Local actor
internal
Network edge
Loopback
yusuf
endpoint
/bin/bash
/var/log/auth.log
Credential Access
T1110.001
INITIAL ACCESS
Foothold established on endpoint
Inbound session accepted from external host on yusuf.
INFERRED
EXECUTION
Adversary code executed on host
Payload spawned on yusuf.
INFERRED
CREDENTIAL ACCESS
T1110.001 · Password Guessing
SSHD brute force trying to get access to the system.
Wazuh rule 5716 (level 10) fired on yusuf.
OBSERVED
AI Triage
LOCAL · QWEN2.5 14B
Qwen2.5 14B is reading the alert...
Analyzing...
Jun 30, 16:39:28
Medium · L8
yusuf
New user account created locally.
T1136.001
—
Jun 30, 16:33:58
Medium · L8
yusuf
New user account created locally.
T1136.001
—
Jun 30, 16:33:57
Medium · L10
yusuf
SSHD brute force trying to get access to the system.
T1110.001
—
Page 1 of 1
Previous
Next
Wazuh connected
Manager: /api/manager


SentinelView
Wazuh SIEM Console
Security Operations Console
LIVE
Manager: /api/manager
Status: ONLINE
Overview
Agents
Alerts
MITRE ATT&CK
Reports
AI Assistant
MITRE ATT&CK Coverage
Techniques observed across the monitored endpoints, grouped by tactic.
Execution
1 technique · 2 alerts
T1059.001
PowerShell
2
HIGH
Persistence
1 technique · 2 alerts
T1136.001
Local Account
2
MEDIUM
Credential Access
2 techniques · 4 alerts
T1110.001
Password Guessing
2
MEDIUM
T1003.001
LSASS Memory
2
CRITICAL
Wazuh connected
Manager: /api/manager


SentinelView
Wazuh SIEM Console
Security Operations Console
LIVE
Manager: /api/manager
Status: ONLINE
Overview
Agents
Alerts
MITRE ATT&CK
Reports
AI Assistant
Reports & Export
Summarize and export alert data for the selected date range.
FROM
Jun 23, 2026 - 17:21
TO
Jun 30, 2026 - 17:21
Export CSV
Export PDF
CRITICAL
2
HIGH
2
MEDIUM
4
LOW
982
Report summary — 990 alerts
RULE ID
DESCRIPTION
SEVERITY
COUNT
40704
Systemd: Service exited due to a failure.
Low
981
5716
SSHD brute force trying to get access to the system.
Medium
2
5902
New user account created locally.
Medium
2
91382
Suspicious PowerShell execution detected (Obfuscation/EncodedCommand).
High
2
92932
LSASS Memory Dump detected. Possible credential theft (Procdump/Mimikatz).
Critical
2
5502
PAM: Login session closed.
Low
1
Wazuh connected
Manager: /api/manager


SentinelView
Wazuh SIEM Console
Security Operations Console
LIVE
Manager: /api/manager
Status: ONLINE
Overview
Agents
Alerts
MITRE ATT&CK
Reports
AI Assistant
SOC Assistant
Ask questions about alerts, MITRE techniques, or response actions. Runs locally — no data leaves the machine.
qwen2.5:14b
What does a T1003.001 alert mean and how do I respond?
Understanding T1003.001 Alert
Description: T1003.001 is an MITRE ATT&CK technique related to "OS Credential Dumping" specifically targeting the Windows SAM (Security Accounts Manager) database. This involves unauthorized access or extraction of stored credentials from the local system.
Wazuh Rule ID: Typically, Wazuh correlates this activity with specific rule IDs that detect attempts to dump credentials via LSASS memory or other methods.
Example Alert
An example alert might look like:
Response Actions
1. Containment:
Isolate the Affected System: Immediately isolate the system from the network to prevent lateral movement.
Disable User Accounts: Temporarily disable user accounts that may have been compromised.
2. Eradication:
Remove Malware/Backdoors: Use antivirus software or other tools to remove any detected malware or backdoor programs.
Update Passwords: Force a password reset for all affected users and ensure strong, unique passwords are used.
3. Recovery:
Restore from Backup: Restore the system from a known good backup if available.
Verify Integrity: Ensure that no malicious changes were made to critical system files or configurations.
4. Post-Incident Actions:
Ask about a MITRE technique, rule ID, or response action...
Send
Wazuh connected
Manager: /api/manager



4.3 Rules Engineering & Threat Mapping (Anas)

As the Rules Engineer, Anas was responsible for translating raw telemetry into actionable, high-fidelity security alerts by writing custom Wazuh decoders and XML rules.

Step 1: Accessing the Ruleset

- SSH'd directly into the AWS Wazuh Manager (`54.83.241.104`).
- Navigated to the custom rules directory: `nano /var/ossec/etc/rules/local_rules.xml`.

Step 2: Engineering Windows Detection Rules (Sysmon)
To detect Masoud's attacks, Anas analyzed Sysmon Event ID 1 (Process Creation) and Event ID 10 (Process Access) and wrote the following strict rules:

```
<group name="windows, sysmon, custom_rules,">
  <!-- Detect Malicious Obfuscated PowerShell -->
  <rule id="100001" level="12">
    <if_group>sysmon_event1</if_group>
    <field name="sysmon.image">powershell.exe</field>
    <field name="sysmon.commandLine">-EncodedCommand|-e |Invoke-Expression|IEX</field>
    <description>Suspicious PowerShell Execution Detected</description>
    <mitre>
      <id>T1059.001</id>
    </mitre>
  </rule>

  <!-- Detect LSASS Memory Dumping -->
  <rule id="100002" level="14">
    <if_group>sysmon_event10</if_group>
    <field name="sysmon.targetImage">lsass.exe</field>
    <field name="sysmon.grantedAccess">0x1010|0x1410|0x1f0fff</field>
    <description>LSASS Memory Dump Attempt Detected</description>
    <mitre>
      <id>T1003.001</id>
    </mitre>
  </rule>
</group>
```

Step 3: Engineering Linux Detection Rules (Auditd/Syslog)

To detect Youssef's attacks, Anas correlated failed authentication logs and user creation events:

```
<group name="linux, custom_rules,">
  <!-- Detect SSH Brute Force (Multiple Failures) -->
  <rule id="100003" level="10" frequency="5" timeframe="60">
    <if_matched_sid>5716</if_matched_sid> <!-- Base SSH Failure Rule -->
    <description>SSH Brute Force Attack Detected</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
  </rule>

  <!-- Detect Local Account Creation (Persistence) -->
  <rule id="100004" level="8">
    <if_sid>5902</if_sid> <!-- Base sudo/command execution rule -->
    <match>useradd|adduser</match>
    <description>Unauthorized Local Account Created</description>
    <mitre>
      <id>T1136.001</id>
    </mitre>
  </rule>
</group>
```

Step 4: Restarting the Manager & Validating MITRE Mappings

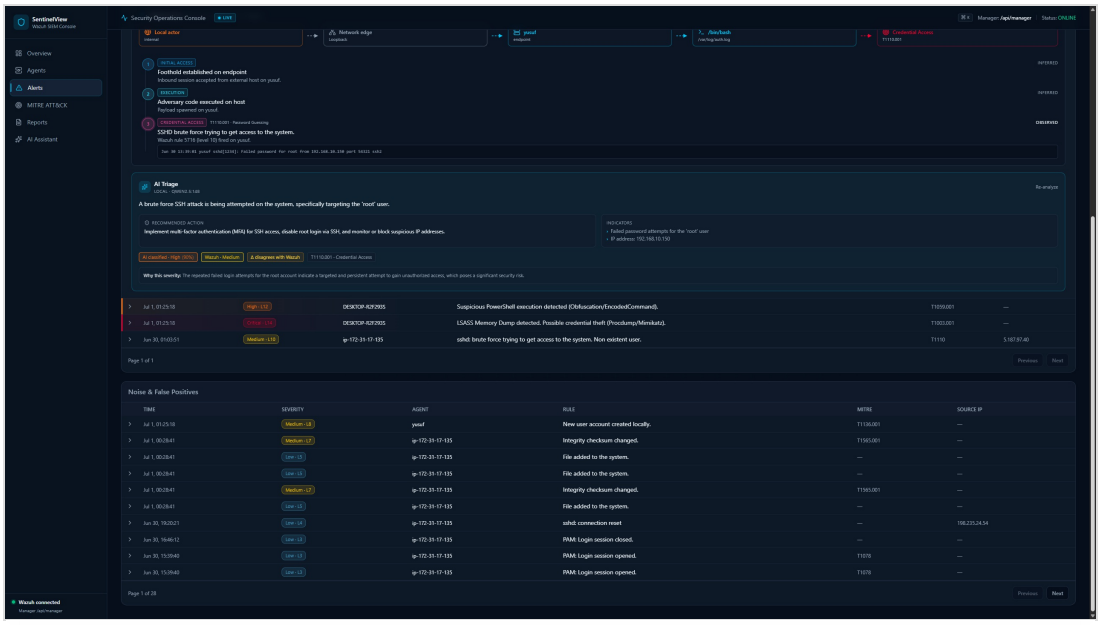
- Restarted the Wazuh Engine to compile the new rules: `sudo systemctl restart wazuh-manager`.
- Validated that the `<mitre><id>` blocks successfully parsed into the JSON alert payload, allowing the SentinelView GUI matrix to instantly interpret the data.

4.4 Artificial Intelligence & Heuristics Integration (Anas)

To separate SentinelView from a standard SIEM, Anas exclusively designed and integrated a specialized AI and heuristic layer.

- **Heuristic Engine:** Developed `aiService.ts` to assign exact mathematical risk coefficients to various MITRE ATT&CK techniques (e.g., heavily weighting Credential Dumping over basic Discovery).

- **True Positive Verification:** Engineered an algorithm that evaluates Wazuh alerts in real-time. If an alert's base severity and custom MITRE Risk Score exceed a specific threshold, the system automatically verifies it and promotes it to the **"AI-Verified Threats (True Positives)"** feed.
- **Noise Filtering:** Alerts falling below the heuristic threshold are filtered and paginated into a separate "Noise" block, keeping the main feed clean.
- **Generative Threat Summarization:** Integrated generative AI models to provide instant, plain-English incident summaries, technical breakdowns, and remediation steps whenever a verified threat is clicked on the dashboard.



5. Final Conclusion

Through seamless collaboration, the **ThreatScopeLab** team engineered a complete, enterprise-grade pipeline. The raw logs generated by **Masoud** and **Youssef** flowed instantly from their local VMs into **Hazem & Abdelrahman's** AWS cloud. **Anas's** custom rules detected the attacks seamlessly, and the telemetry was piped securely into the custom SentinelView dashboard built by **Anas and Adham**, where the AI validated the threats with pinpoint accuracy.